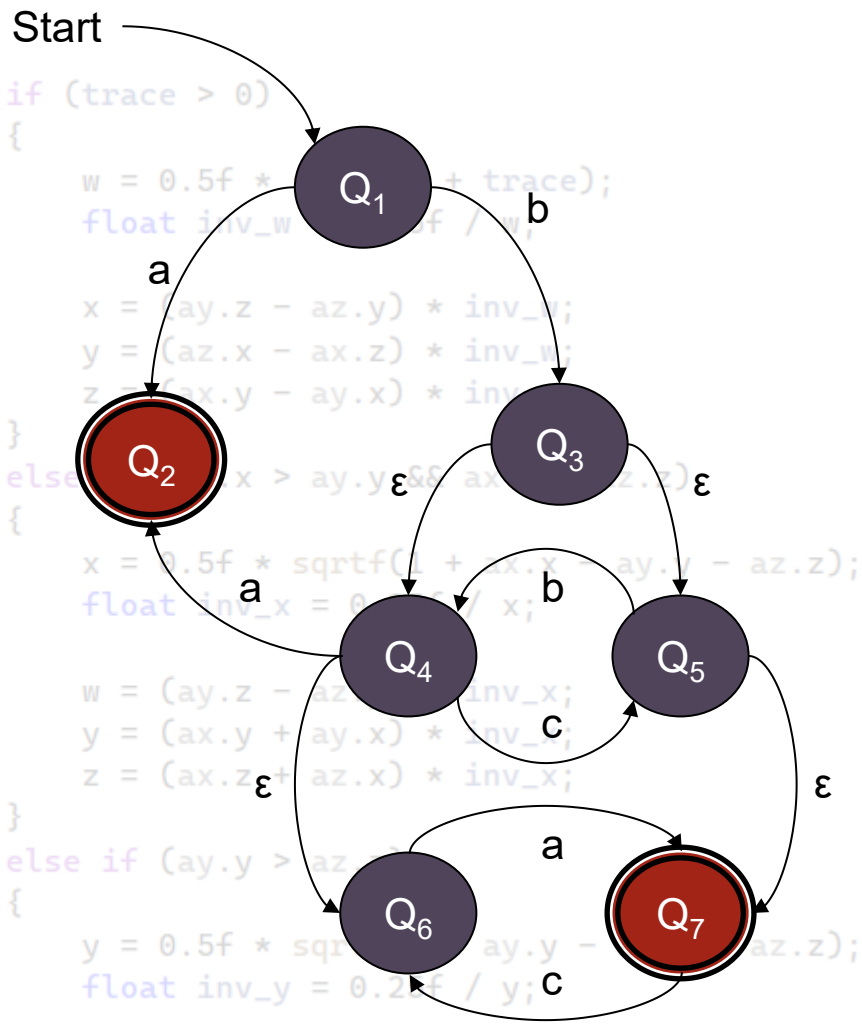
Algorithmen und Datenstrukturen

6 – Sequentielle Container und Adapter



Tobias Werner M. Sc.
Uni Bayreuth - Angewandte Informatik 3
Leibniz-FH Hannover
Duale Hochschule Sachsen
tobias.werner@uni-bayreuth.de

Einleitung

Heranführung

- Was ist der Unterschied im Vorgehen zwischen...
  ... einer Warteschlange an einer Supermarktkasse und
  ... einem Stapel schwarze T-Shirts im Kleiderschrank?

- Gegeben eine große Schachtel mit Cookies.
  Wie findet man am schnellsten den besten Cookie?

- Was heißt O(n)?
- Was heißt O(1)?

Kompetenzen

- Array und Liste als sequentielle Container bewerten, anwenden und implementieren
- Heaps verwenden und als Array-Adapter realisieren
- Queues und Stacks verwenden und als Adapter über Array oder Liste realisieren

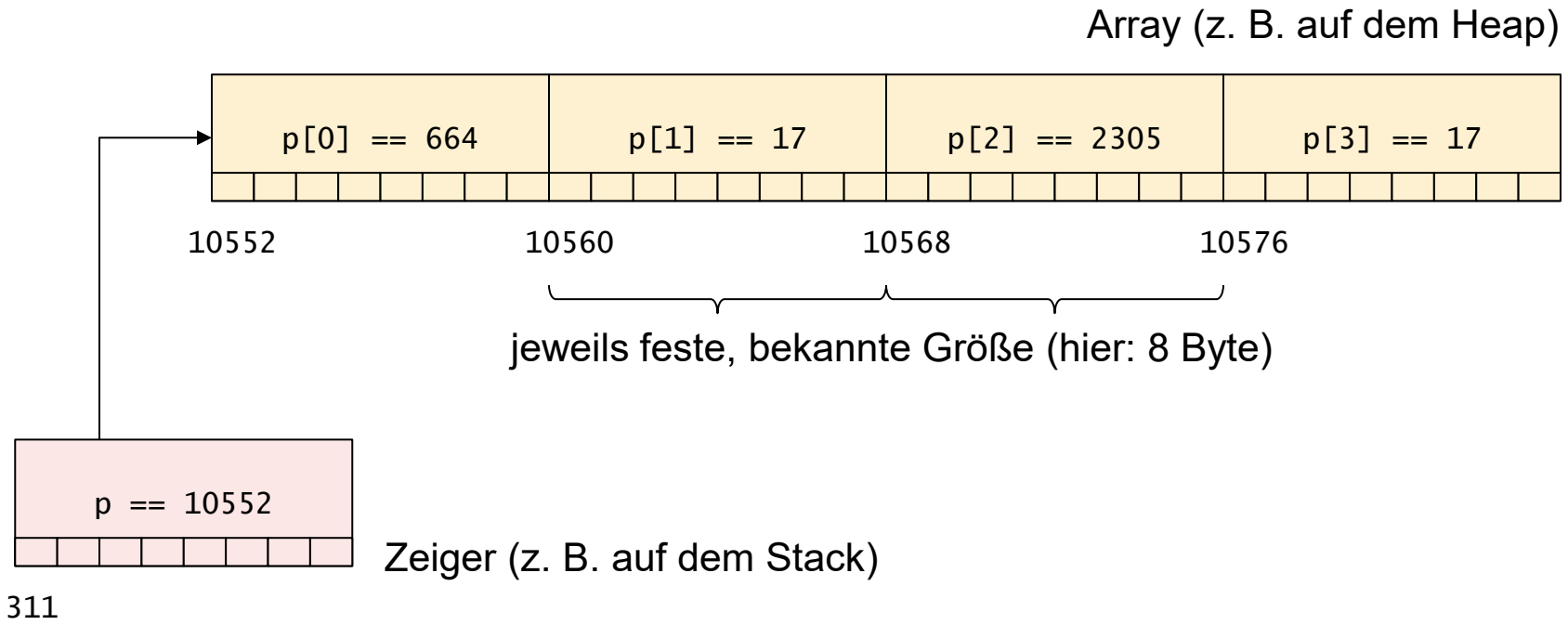
Vortragsübersicht

6 – Sequentielle Container und Adapter

- Arrays
- Listen
- Heaps
- Suche in Arrays
- Stacks und Queues

Arrays (1)

Wiederholung



Zeigerarithmetik ermöglicht Zugriff auf mehrere, im Speicher dichtauf liegende Elemente über einen Zeiger auf das erste Element und einen gegebenen Index

→ ein *Array* ist ein Container, der einen (i. d. R.) schnellen sowie wahlfreien Zugriff auf sequentielle Elemente über einen gegebenen Index ermöglicht

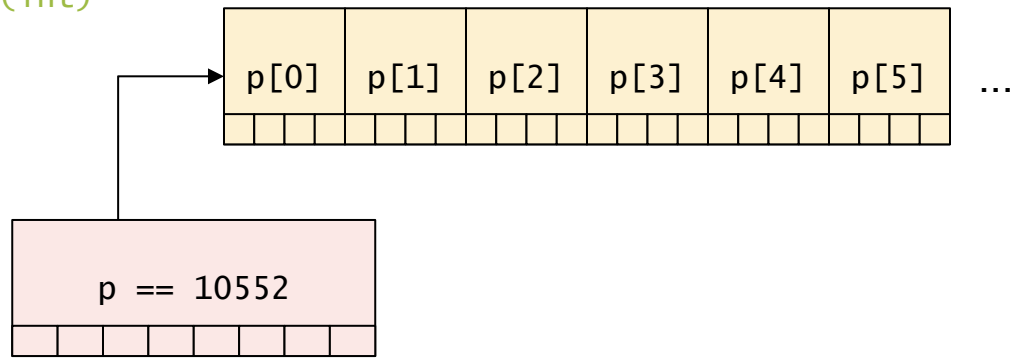
Arrays (2)

Grundoperationen auf Arrays

Zugriff per Index:

verwende die Zeigerarithmetik, um einen Zeiger auf das Element am Index zu erzeugen, dereferenziere den Zeiger zum Zugriff

```
int *p = new int[4000000];
// ... initialisiere Werte ...
// in der Theorie gleiche Laufzeit,
// Rechnung jeweils + offset * sizeof(int)
std::cout << *(p + 2);
std::cout << p[3999999];
```



→ Zugriffszeit hängt nicht vom Wert des Indexes ab, sofern es eine obere Schranke für einen maximalen Index gibt (z. B. feste Bitbreite der Zahlen)

→ Zugriffszeit über Index ist in O(1)

→ **ACHTUNG: in der Praxis hat z. B. Caching der CPU einen deutlichen Einfluss!**

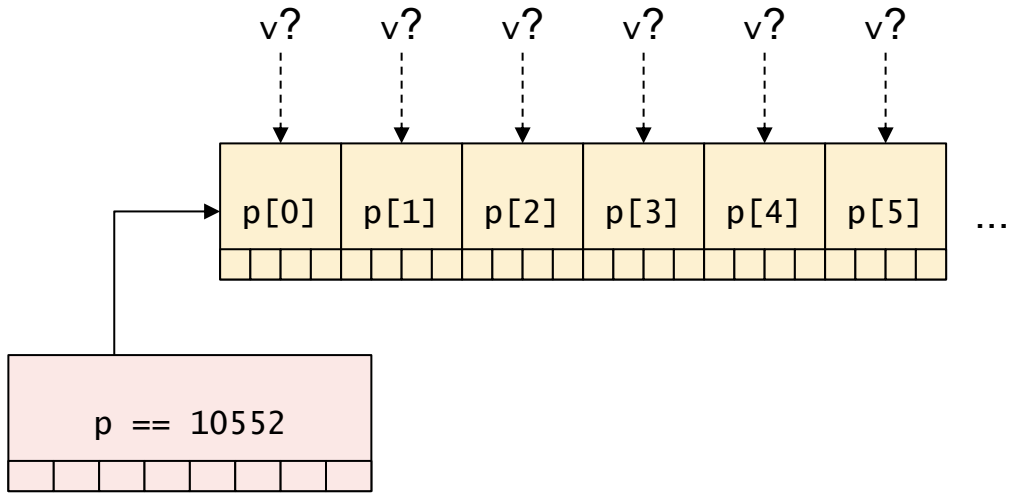
Arrays (3)

Suche nach gegebenem Element mit Kriterium:

gehe alle Indizes der Reihe nach durch und prüfe, ob das Element das Kriterium erfüllt

(z. B. hat das Element einen bestimmten Wert gespeichert?)

```
int n = /* ... einlesen ... */;
// Größe n geht nur für dynamische Arrays
int *p = new int[n];
// ... initialisiere Werte ...
// Laufzeit hängt linear von n ab.
int v = /* gesuchter Wert */;
int result = -1;
for (int i = 0; i < n; ++i)
{
    if (p[i] == v)
        { result = i; break; }
}
```



im schlimmsten Fall sind alle Elemente auf das Kriterium zu prüfen,

weil man (außer die Reihenfolge) nichts über die Elemente weiß

→ Worst-Case-Suchdauer ist in O(*n*)

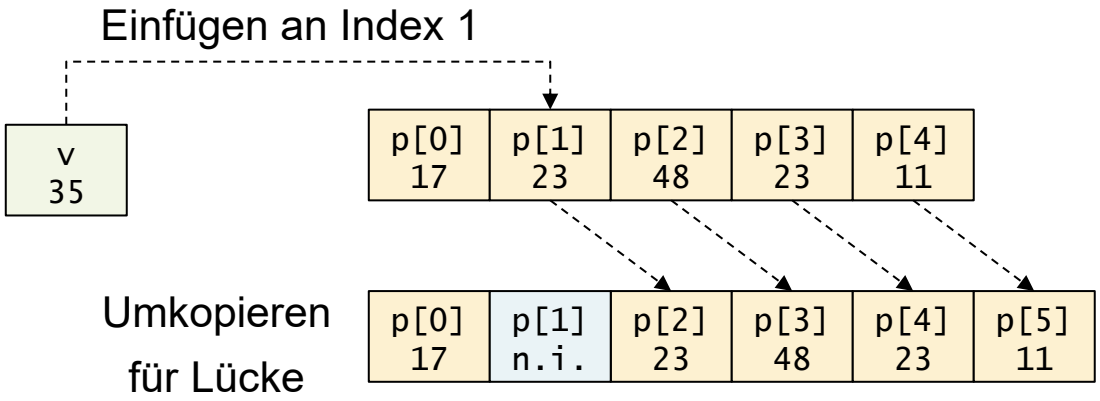
Arrays (4)

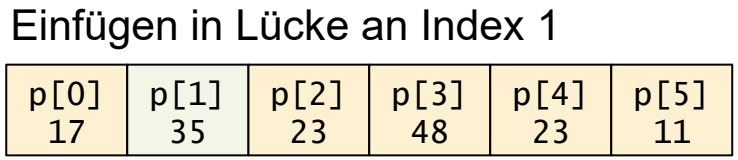
Einfügen in Arrays

Einfügen mit Erhalt der Reihenfolge:

gegeben: einzufügendes Datum, Einfügestelle als Index

gesucht: Array mit zusätzlichem Element an Einfügestelle, wobei die Reihenfolge bestehender Elemente zueinander erhalten bleibt





Vorgehen: schaffe eine Lücke für das neue Element, füge den neuen Wert in die Lücke ein

→ schlimmstenfalls Verschieben aller Array-Einträge nötig

→ Iteratoren werden (i. d. R. ohnehin durch Verschieben des ganzen Arrays) invalidiert

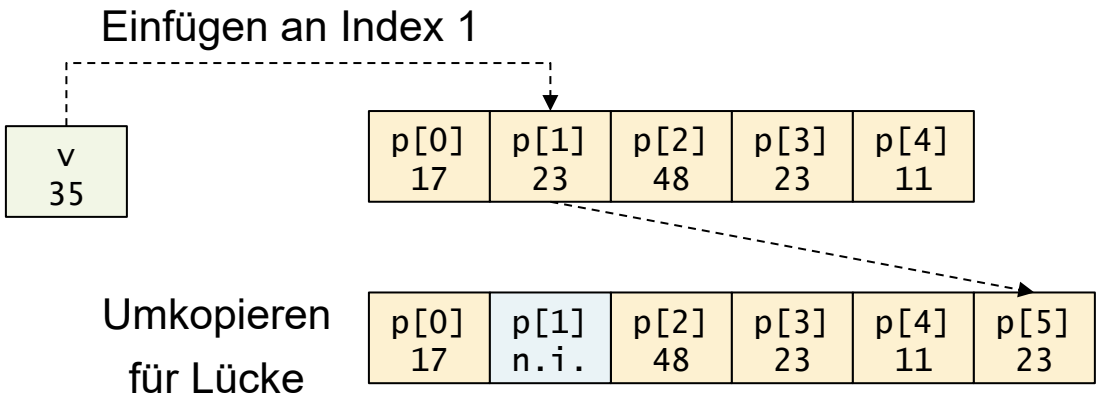
→ Laufzeit O(n)

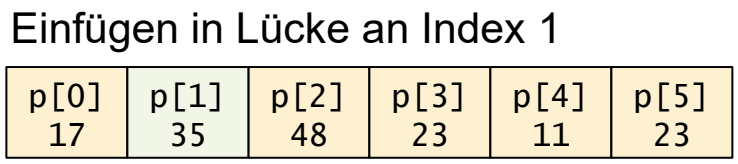
Arrays (5)

Einfügen ohne Erhalt der Reihenfolge:

gegeben: einzufügendes Datum, Einfügestelle als Index

gesucht: Array mit zusätzlichem Element an Einfügestelle, wobei die Reihenfolge bestehender Elemente zueinander nicht erhalten bleiben braucht





Vorgehen: schaffe eine Lücke für das neue Element, füge den neuen Wert in die Lücke ein

→ Verschieben von nur einem einzigen Element nötig

→ ohne Umkopieren werden nur Iteratoren auf das verschobene Element invalidiert

→ Laufzeit O(1)

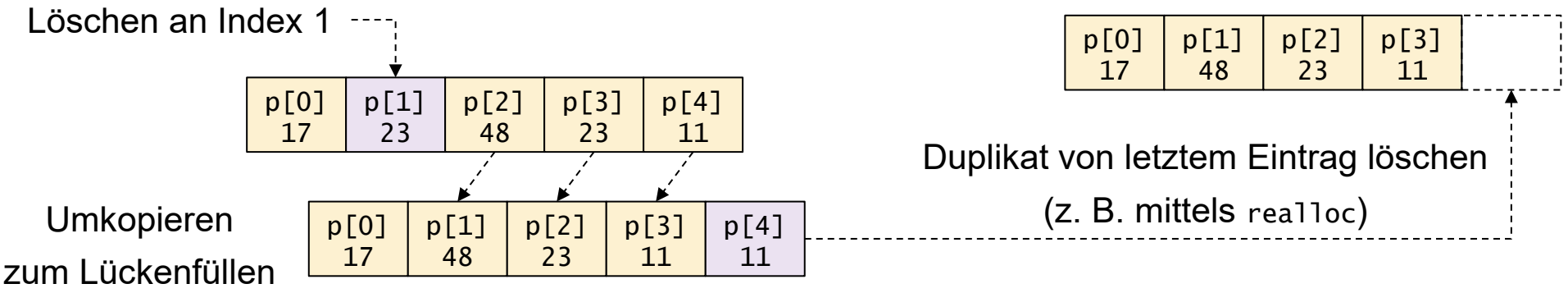
Arrays (6)

Löschen in Arrays

Löschen mit Erhalt der Reihenfolge:

gegeben: Array, Index für zu löschendes Element

gesucht: Array ohne das zu löschende Element, wobei die Reihenfolge nicht gelöschter Elemente zueinander erhalten bleibt



Vorgehen: verschiebe alle Elemente im Array nach links, um die durch das Löschen entstandene Lücke zu schließen

→ schlimmstenfalls Verschieben aller Array-Einträge nötig

→ Iteratoren werden (evtl. ohnehin durch Verschieben des ganzen Arrays) invalidiert

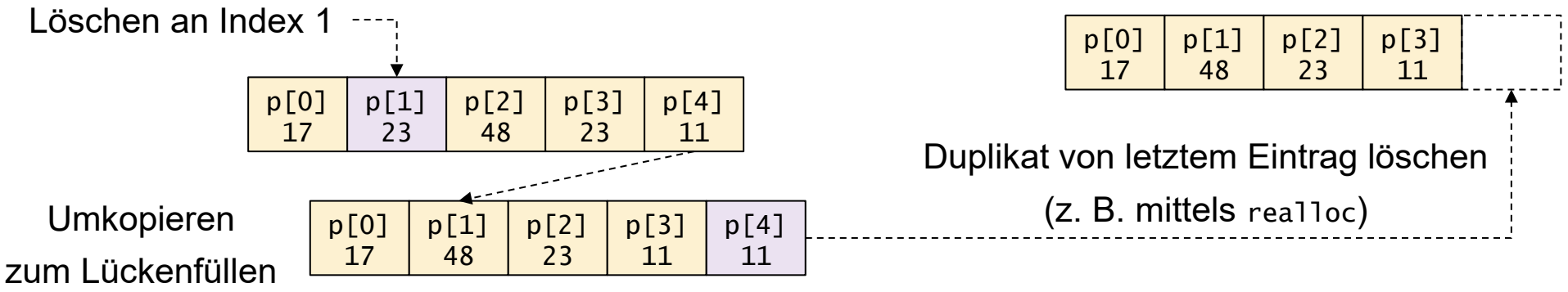
→ Laufzeit O(*n*)

Arrays (7)

Löschen ohne Erhalt der Reihenfolge:

gegeben: Array, Index für zu löschendes Element

gesucht: Array ohne das zu löschende Element, wobei die Reihenfolge bestehender Elemente zueinander nicht erhalten bleiben braucht



Vorgehen: verwende immer das letzte Element des Arrays zum Lückenfüllen

→ Verschieben von nur einem einzigen Element nötig

→ ohne Umkopieren werden nur Iteratoren auf das verschobene Element invalidiert

→ Laufzeit O(1)

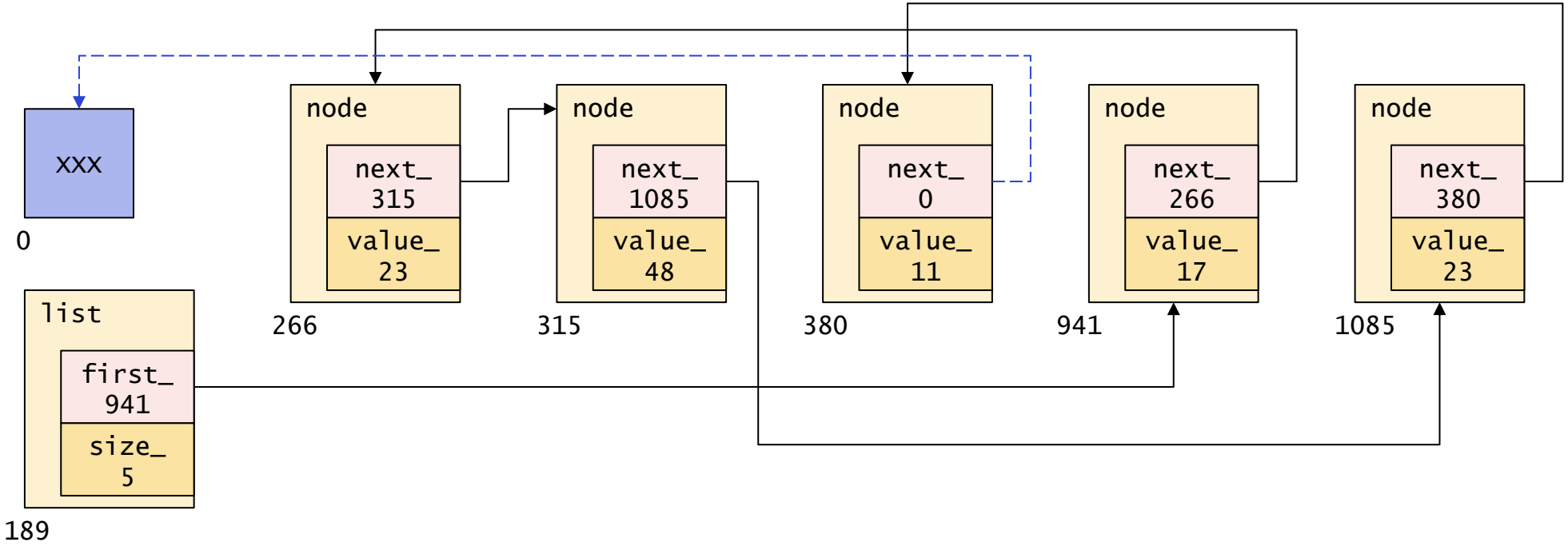
Vortragsübersicht

6 – Sequentielle Container und Adapter

- Arrays
- Listen
- Heaps
- Suche in Arrays
- Stacks und Queues

Listen (1)

Grundkonzept



eine *Liste* ist ein Container, der seine Elemente über sequentielle Verweise von einem Element zum nächsten Element abspeichert

→ häufig werden *Knoten* (Zusatzcontainer, engl. node) für Elemente und Verweise benutzt

→ Verweise sind häufig Zeiger (alternativ wären z. B. Indizes in ein Array möglich)

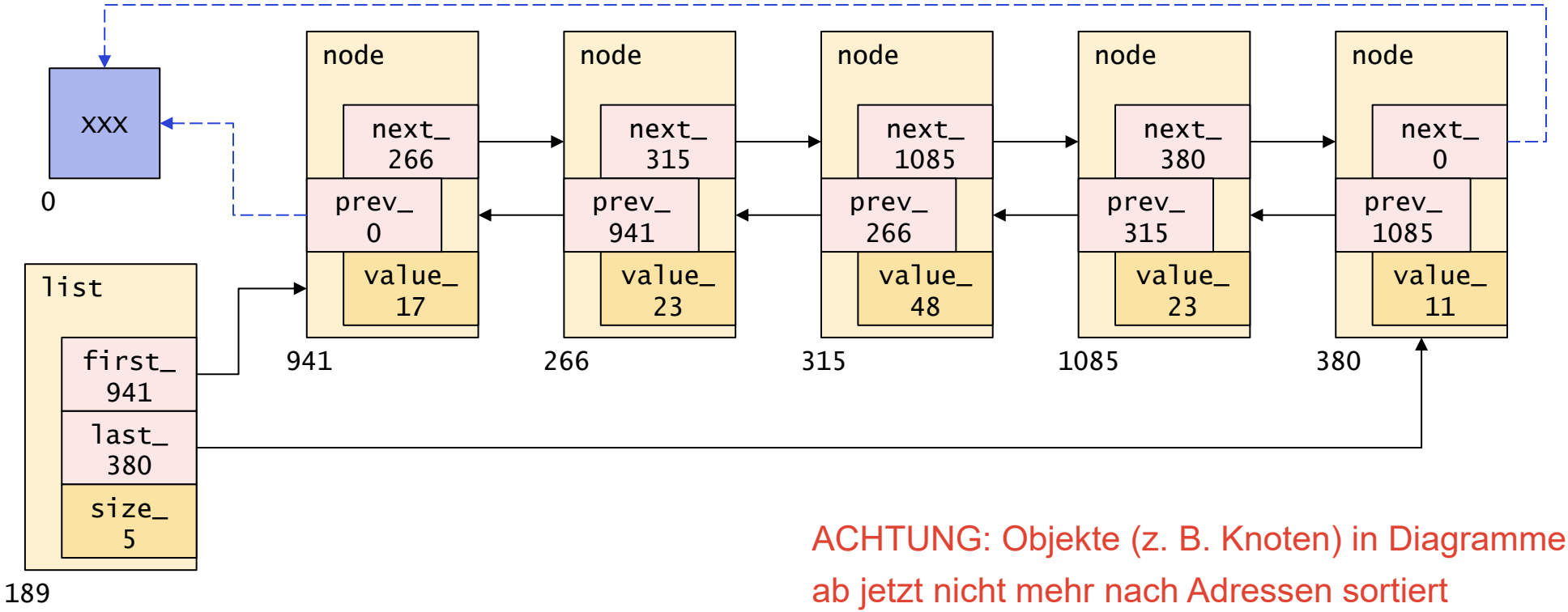
→ der Container merkt sich zumindest einen Verweis auf das erste Element

Listen (2)

Varianten von Listen

→ *singly linked list* (dt. *einfach verkettete Liste*):
nur erstes Element und Nachfolger merken (vgl. vorherige Folie, selten sinnvoll)

→ *doubly linked list* (dt. *doppelt verkettete Liste*):
erstes und letztes Element sowie Vorgänger und Nachfolger merken (übliche Variante)



ACHTUNG: Objekte (z. B. Knoten) in Diagrammen ab jetzt nicht mehr nach Adressen sortiert

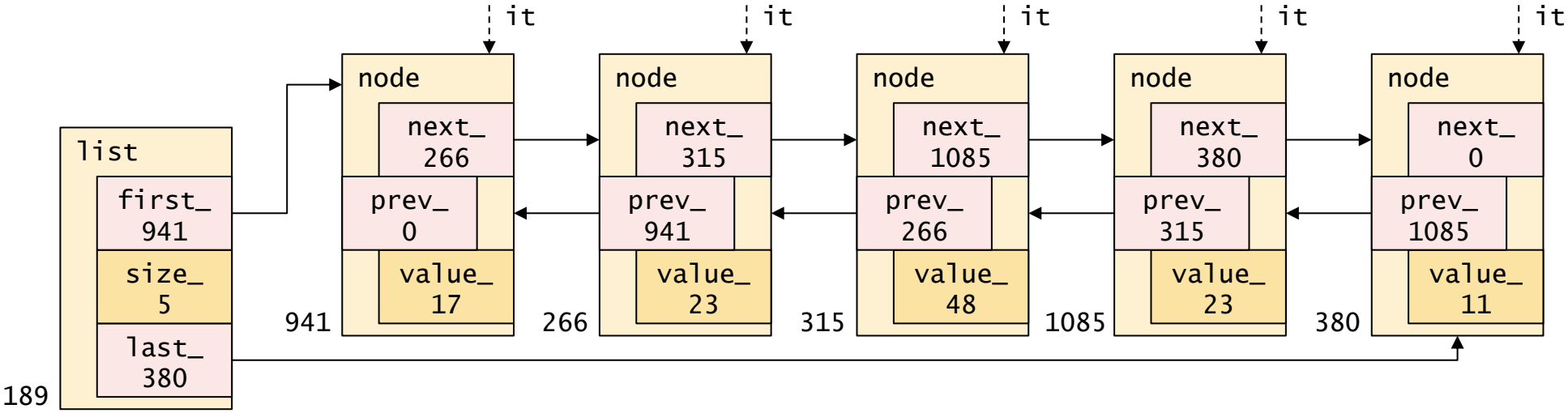
Listen (3)

Grundoperationen auf Listen

Zugriff auf Element per Index:

alle Knoten durchgehen und zählen, bis das gesuchte Element gefunden ist

```
int i = /* zu suchender Index */, ni = 0;
std::list<int> l( /* ... Liste befüllen .. */ );
std::list<int>::iterator it = l.begin();
while (i != ni && it != l.end()) { ++it; ++ni;  }
std::cout << it->value_;
```



schlimmstenfalls muss man alle Knoten (oder die Hälfte davon) durchgehen

→ Laufzeit O(n) = O(n / 2)

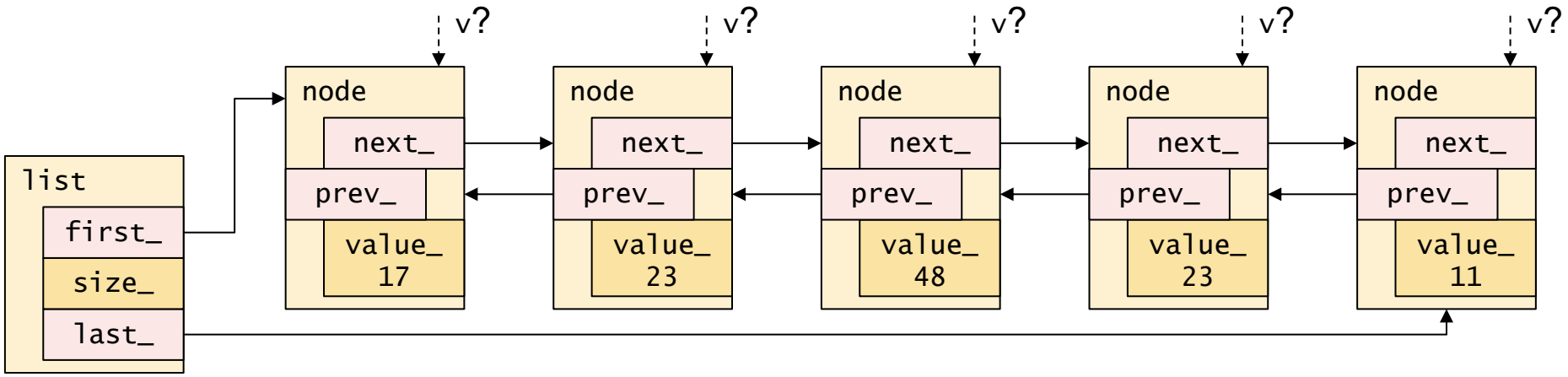
Listen (4)

Suche nach gegebenem Element mit Kriterium:

alle Elemente durchgehend und auf Kriterium prüfen

(z. B. hat das Element einen gegebenen Wert?)

```
std::list<int> l({ /* ... initialisiere Werte ... */ });
// Laufzeit hängt linear von n ab.
int v = /* gesuchter Wert */;
std::list<int>::iterator result(l.begin());
for (; result != l.end(); ++result)
    { if (*result == v) break; }
```



schlimmstenfalls muss man alle Knoten durchgehen

→ Laufzeit O(*n*)

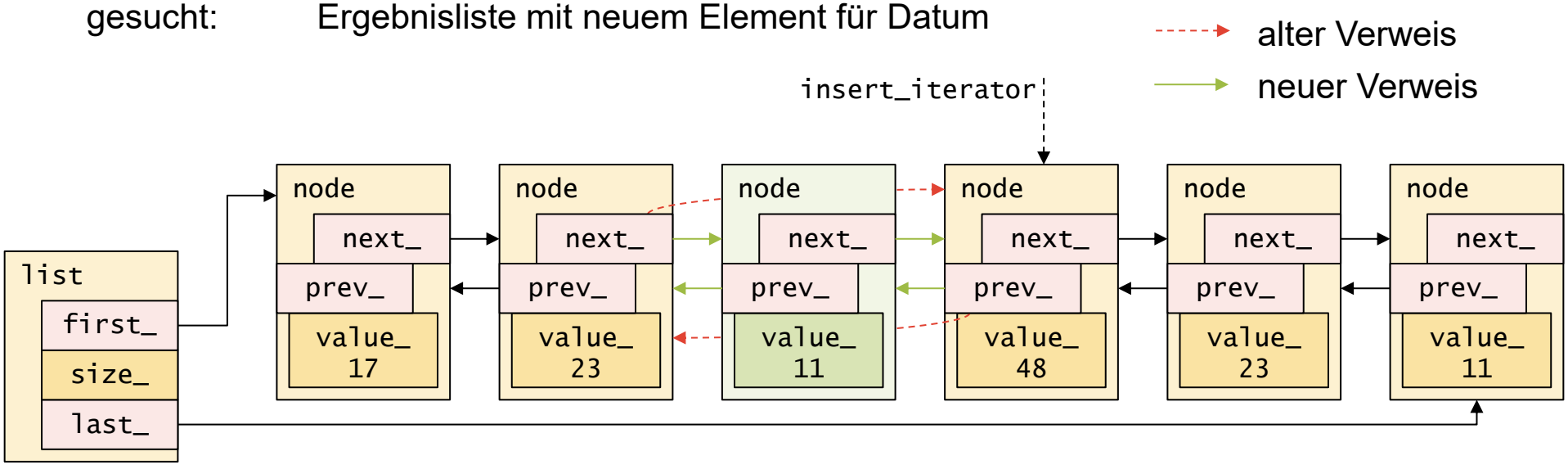
Listen (5)

Einfügen in Listen

Einfügen an beliebiger Stelle:

gegeben: bestehende Liste, einzufügendes Datum, Einfügestelle als Iterator

gesucht: Ergebnisliste mit neuem Element für Datum



Vorgehen: neues Element per Umhängen von Zeigern in die Liste einfügen,
bei Bedarf Anfangs- und Endzeiger anpassen

→ Aufwand unabhängig von Listenlänge

→ Laufzeit O(1)

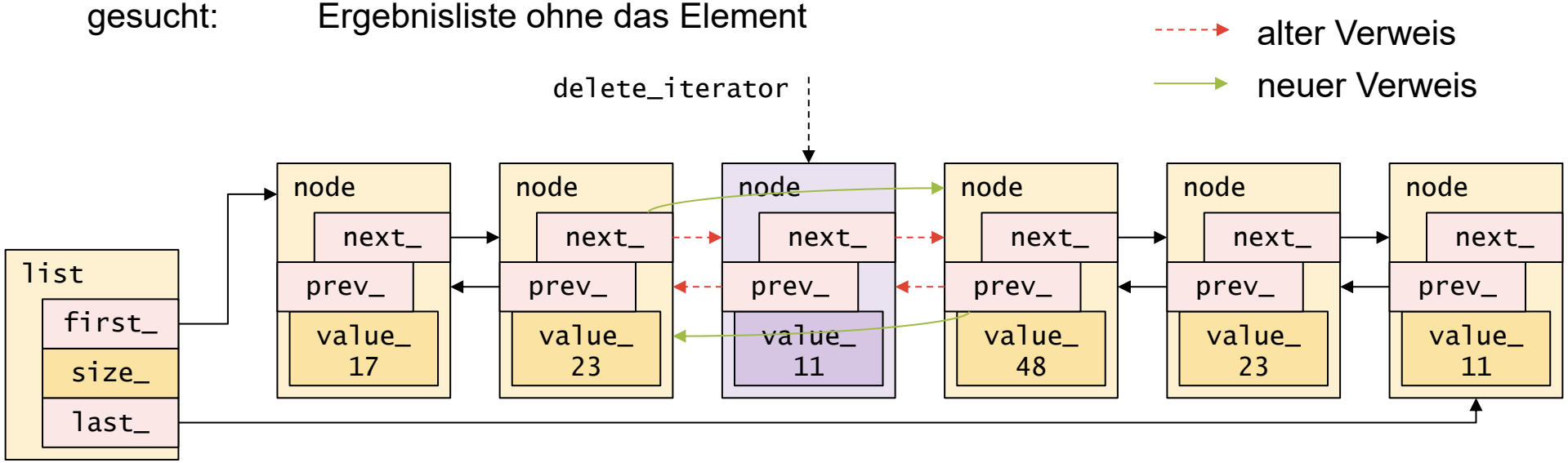
Listen (6)

Löschen in Listen

Löschen an beliebiger Stelle:

gegeben: bestehende Liste, Iterator auf zu löschendes Element

gesucht: Ergebnisliste ohne das Element



Vorgehen: zu löschendes Element über Iterator aus der Liste aushängen,
bei Bedarf Anfangs- und Endzeiger anpassen, ggf. Speicher freigeben

→ Aufwand unabhängig von Listenlänge

→ Laufzeit O(1)

Listen (7)

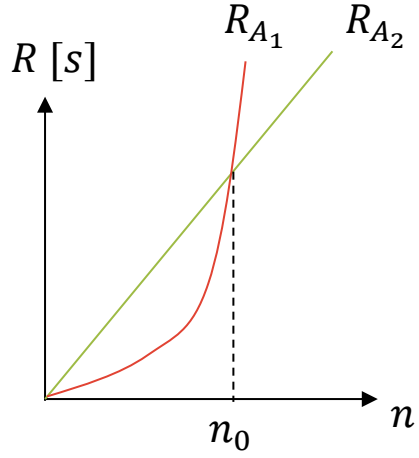
Bedeutung der Komplexität

Praxiserfahrung:

ohne schlaue Optimierung braucht ein Algorithmus auf einem Container mal mehr,
mal weniger Rechenoperationen pro Element

→ für große Anzahl an Elementen n im Container dominiert aber ein anderer Einflussfaktor

→ die Komplexität (als O-Notation) rechnet den konstanten Einfluss heraus
und betrachtet nur das Verhalten für große n



Beispiel:

zwei Algorithmen A_1 und A_2 mit Gesamtrechenzeit R_{A_1} bzw. R_{A_2},
etwa $R_{A_1} = 2n^2$ und $R_{A_2} = 5n$

→ A_1 braucht pro Element weniger Rechenoperationen als A_2

→ für kleine Anzahl n an Elementen gilt deshalb $R_{A_1}(n) < R_{A_2}(n)$

→ trotzdem skaliert A_1 schlechter als A_2, es gibt also ein n_0, ab dem $R_{A_1}(n) > R_{A_2}(n)$

→ für große n ist A_2 (hier: O(n)) besser als A_1 (hier: O(n^2))

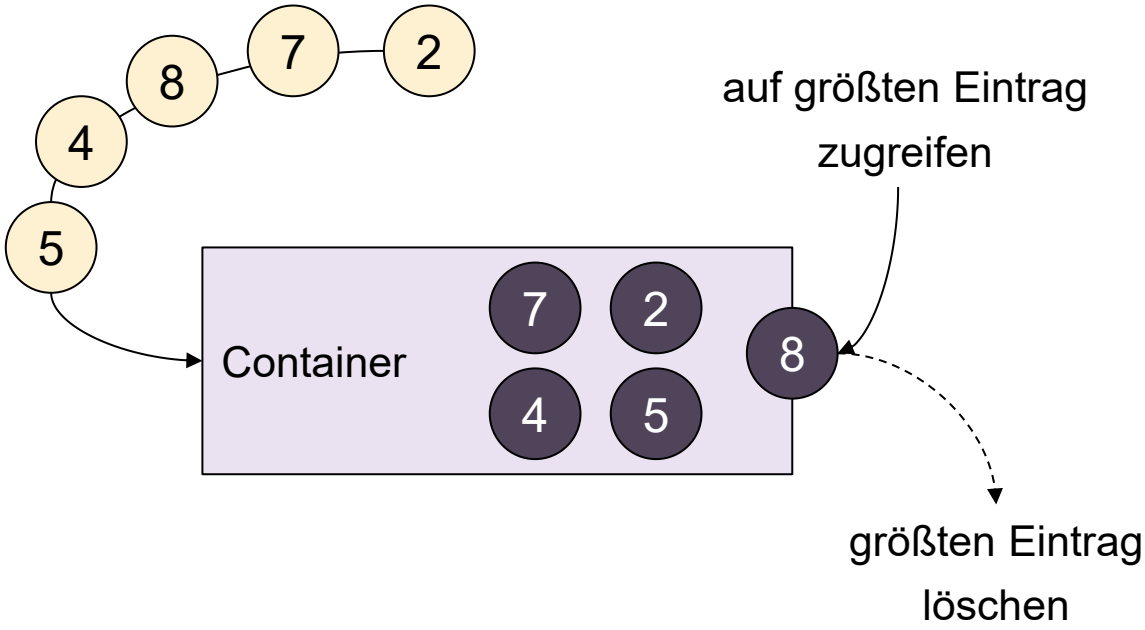
Vortragsübersicht

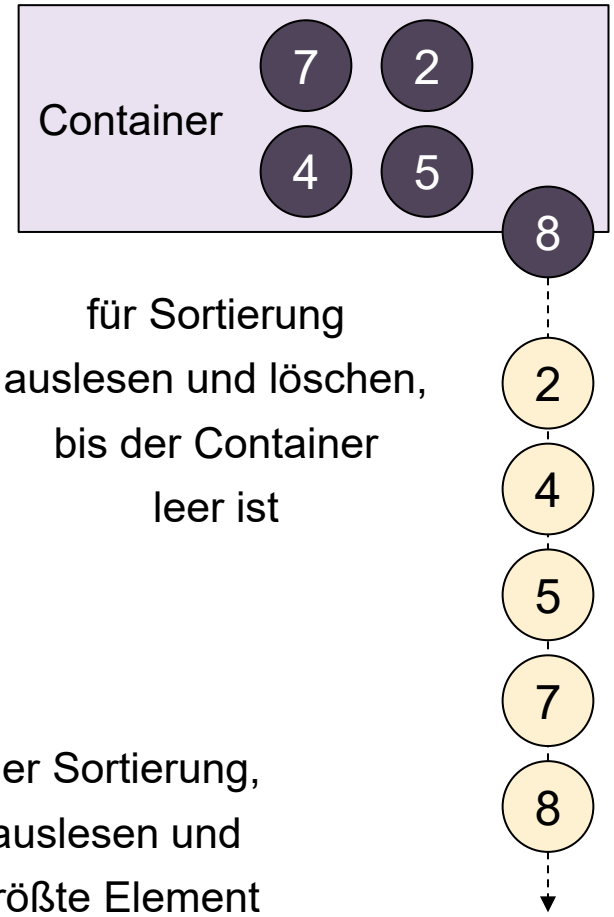
6 – Sequentielle Container und Adapter

- Arrays
- Listen
- Heaps
- Suche in Arrays
- Stacks und Queues

Heaps (1)

Grundkonzept



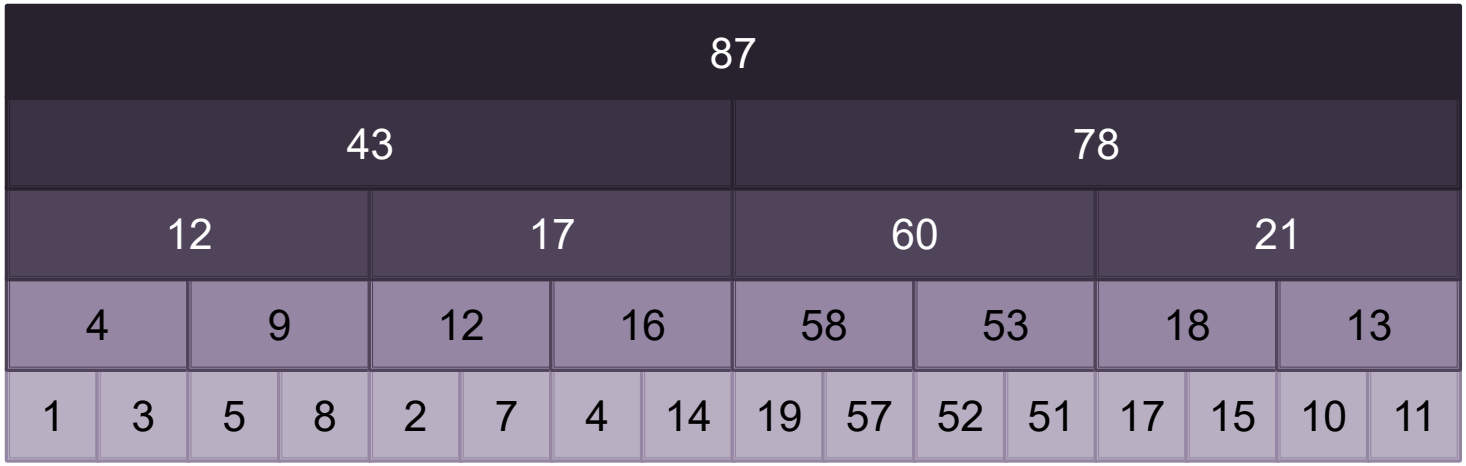


Praxisbeobachtung:

- manchmal interessiert man sich nicht für eine Reihenfolge oder Sortierung, man möchte aber zumindest ein größtes Element kennen, auslesen und ggf. aus dem Container entfernen (derart, dass das dann größte Element weiterhin bekannt ist)
  → einen solchen Container kann man zum Sortieren verwenden

Heaps (2)

Heap



der *Heap* (dt. *Halde*) ist ein Container, der seine Elemente derart sortiert,
dass wichtigere Elemente bezüglich einem Sortierkriterium weiter oben liegen

→ *Min-Heap*: nach oben hin werden die Einträge kleiner, ganz oben steht das Minimum

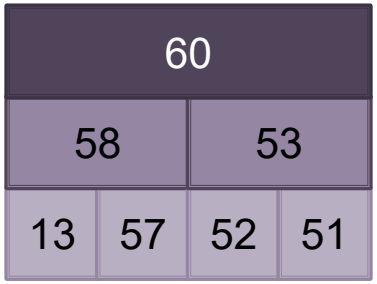
→ *Max-Heap*: nach oben hin werden die Einträge größer, ganz oben steht das Maximum

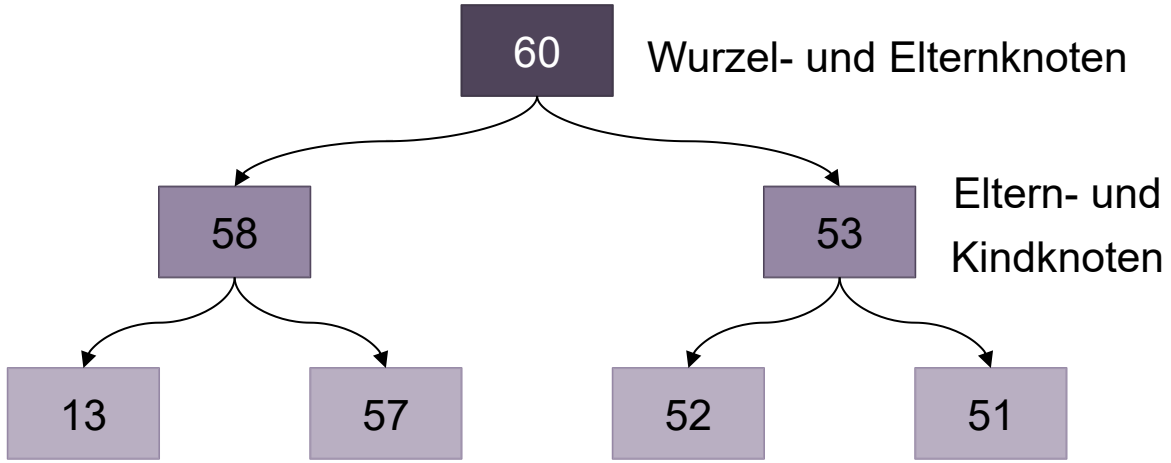
→ abgesehen von der Sortierung nach oben können die Elemente (z. B. auf einer Ebene)
in einer beliebigen Reihenfolge vorliegen

ACHTUNG: nicht mit dem Heap für die Speicherverwaltung verwechseln!

Heaps (3)

Heaps als Bäume





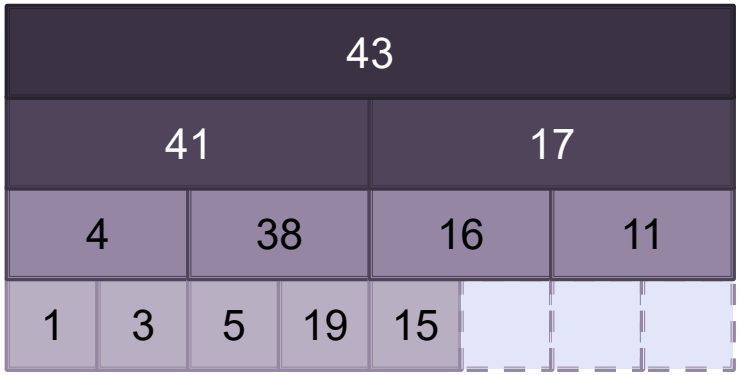
der Heap lässt sich konzeptuell als eine Art von (*Binär-*)*Baum* verstehen:

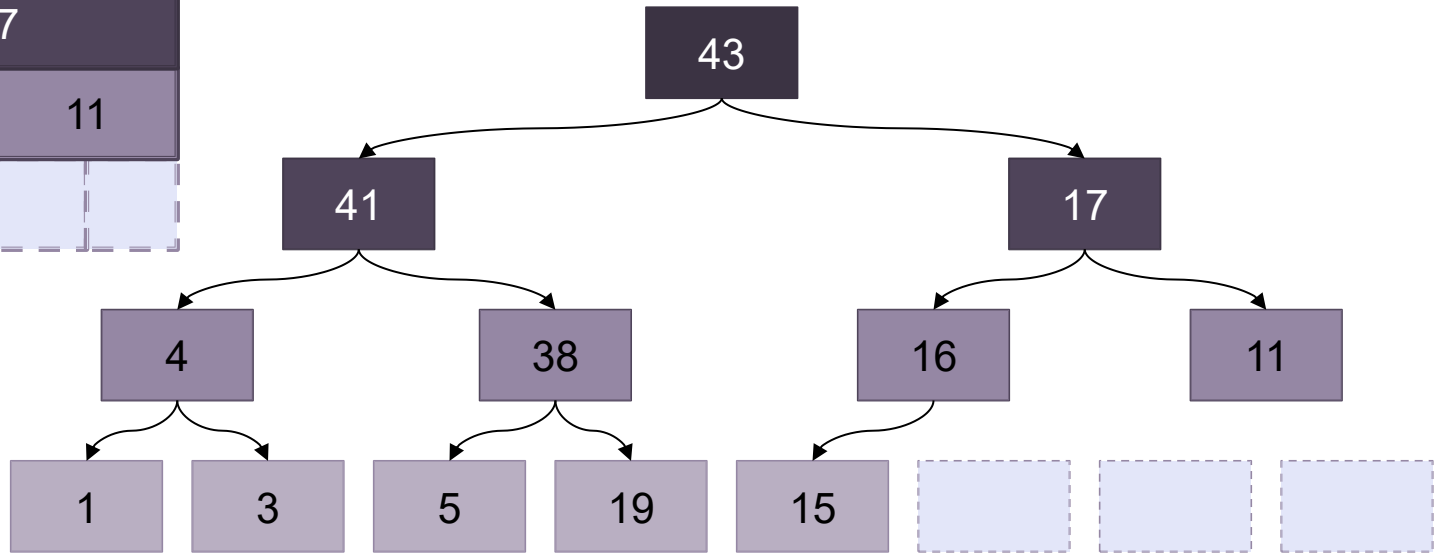
- ein Baum besteht aus *Knoten* mit Verweisen
- jeder Knoten speichert ein Element des Containers
- es gibt exakt einen *Wurzelknoten* (kurz *Wurzel*), das ist der oberste Knoten
- *Elternknoten* (kurz *Eltern*) verweisen auf jeweils ein bis zwei *Kindknoten* (kurz *Kinder*)
- *Blattknoten* (kurz *Blätter*) haben keine Kindknoten

→ andere Bäume in späterer VL-Einheit

Heaps (4)

Umsetzung als Adapter für Arrays





Beobachtung:

im Heap sind nach schrittweisem Befüllen alle Reihen bis auf die letzte Reihe vollständig gefüllt

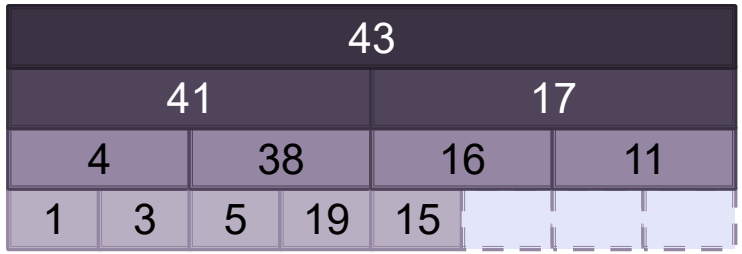
→ dann gibt es ggf. Elternknoten mit nur einem Kind sowie Blattknoten auf höherer Ebene

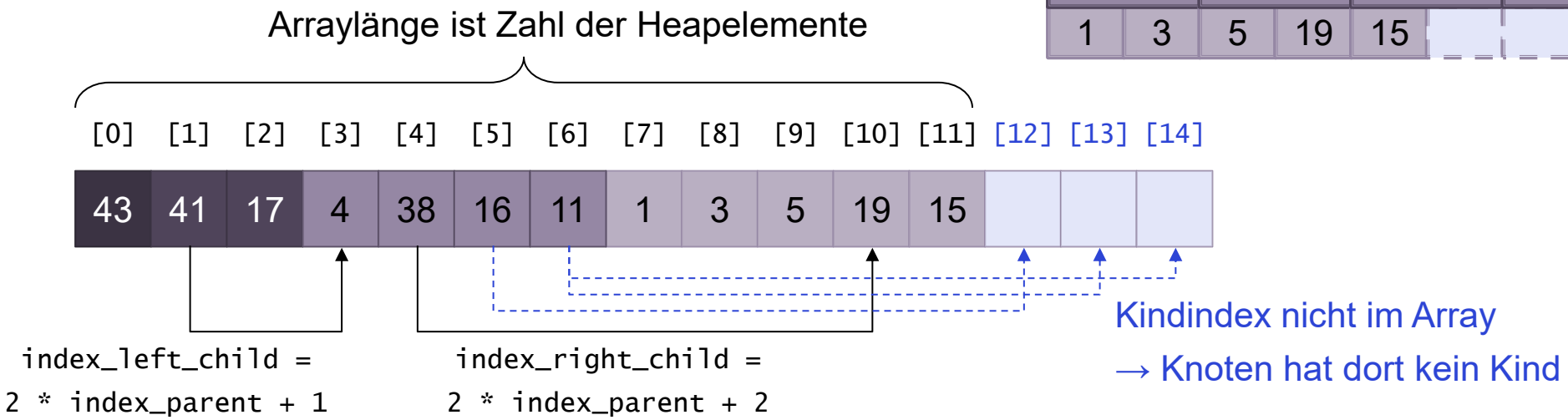
→ bei n vollen Schichten hat der Heap $1 + 2 + 4 + 8 + \cdots = \sum_{i=0}^{n-1} 2^i = 2^n - 1$ Elemente

(vgl. *endliche geometrische Reihe*, Beweis z. B. mit vollständiger Induktion)

Heaps (5)

Umsetzung als Adapter für Arrays





Optimierungsidee:

auch beim Array ist der Standardfall, dass Einträge hinten dazukommen,

und dass das Array innendrin lückenlos ist

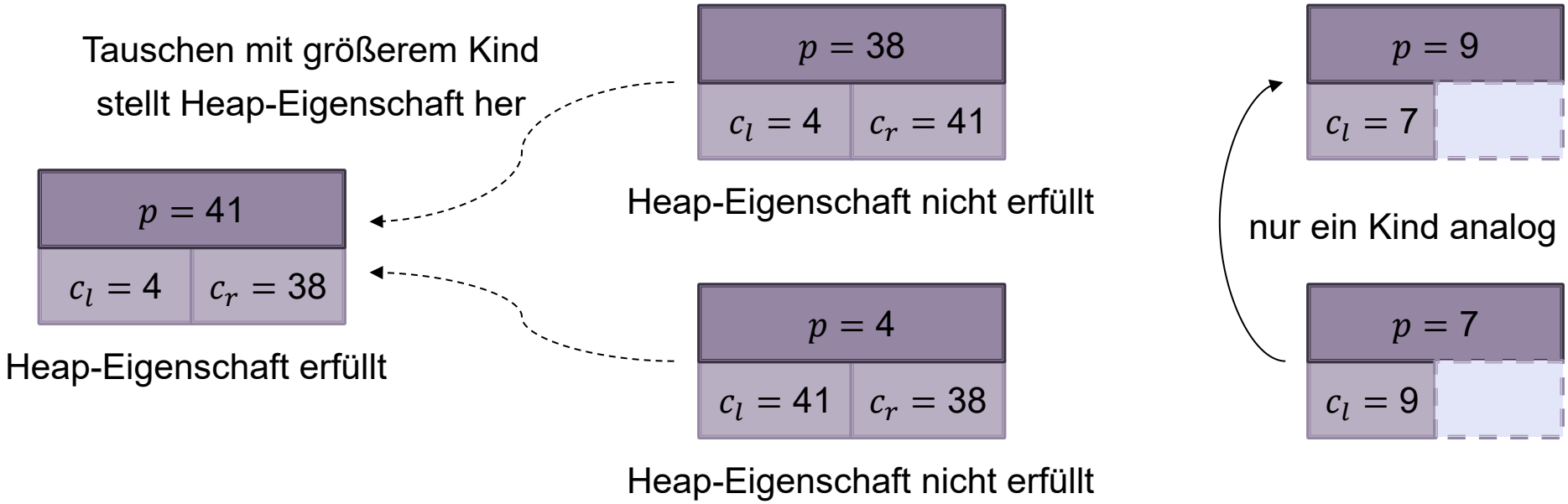
→ speichere den Heap bzw. dessen Knoten in einem Array ab

→ Verweise zwischen Knoten braucht man nicht ausdrücklich als Zeiger abspeichern,

sondern sie sind implizit über den Index von (Eltern-)Knoten im Array gegeben

Heaps (6)

Heapeigenschaft



Definition *Heap-Eigenschaft*:

ein Element p (z. B. eine Zahl) für einen Elternknoten und zwei Elemente c_l und c_r für Kindknoten erfüllen die Heap-Eigenschaft, wenn $p > c_l$ und $p > c_r$

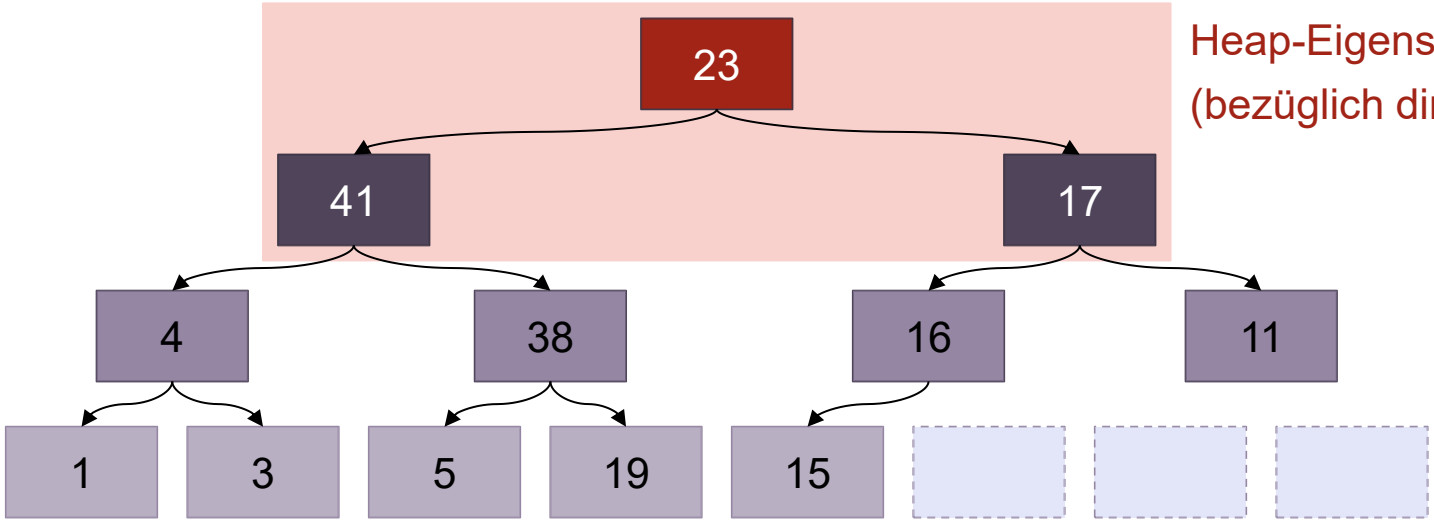
→ falls drei Zahlen die Heap-Eigenschaft nicht erfüllen, kann man diese durch Tauschen der Zahlen immer herstellen

→ analog für ein Element p und einen einziges c_l

→ ein einziges c_r kann bei Füllen von links nach rechts nicht auftreten

Heaps (7)

Heapeigenschaft von oben nach unten herstellen



Heap-Eigenschaft in Wurzel verletzt
(bezüglich direkten Kindern)

Gedankenspiel:

angenommen, **nur** in der Wurzel steht ein Element, welches die Heap-Eigenschaft bezüglich seiner Kinder nicht erfüllt, der restliche Heap ist OK

→ Wie stellt man die Heapeigenschaft so einfach wie möglich wieder her?

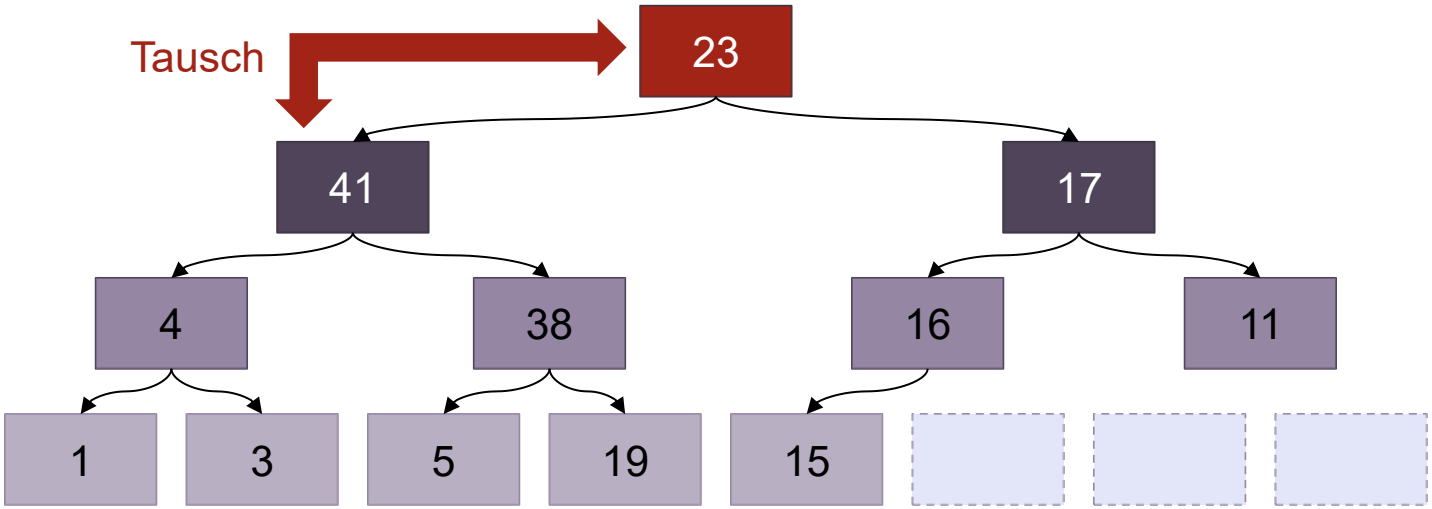
→ "*sift down*": repariere die Heapeigenschaft Stück für Stück in Richtung Blatt
durch Tausch mit dem größeren Kind und Rekursion im geänderten Ast

→ höre auf, wenn Heapeigenschaft an einem Teilbaum schon erfüllt ist

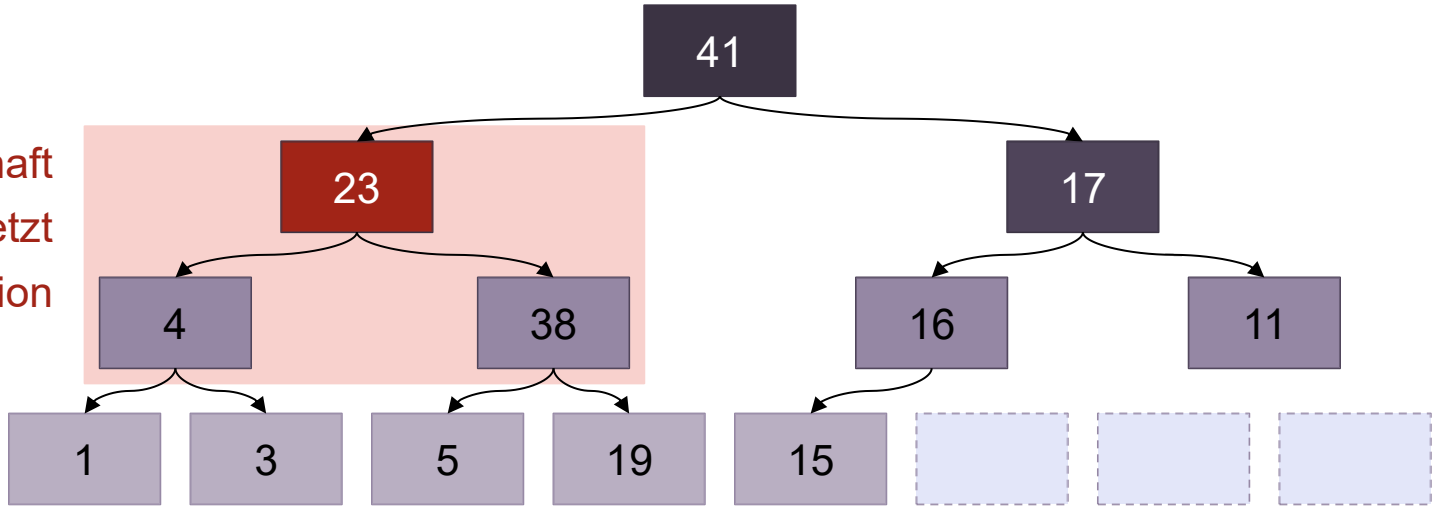
Heaps (8)

Beispiel:

1. Schritt

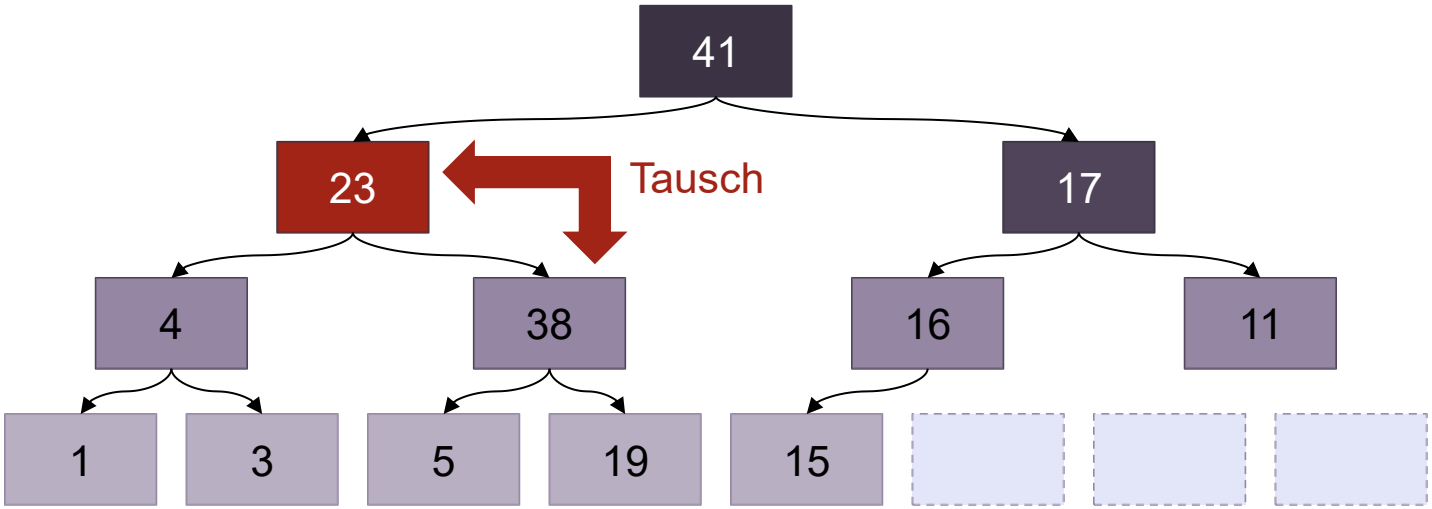


Heap-Eigenschaft in Kind verletzt → Rekursion

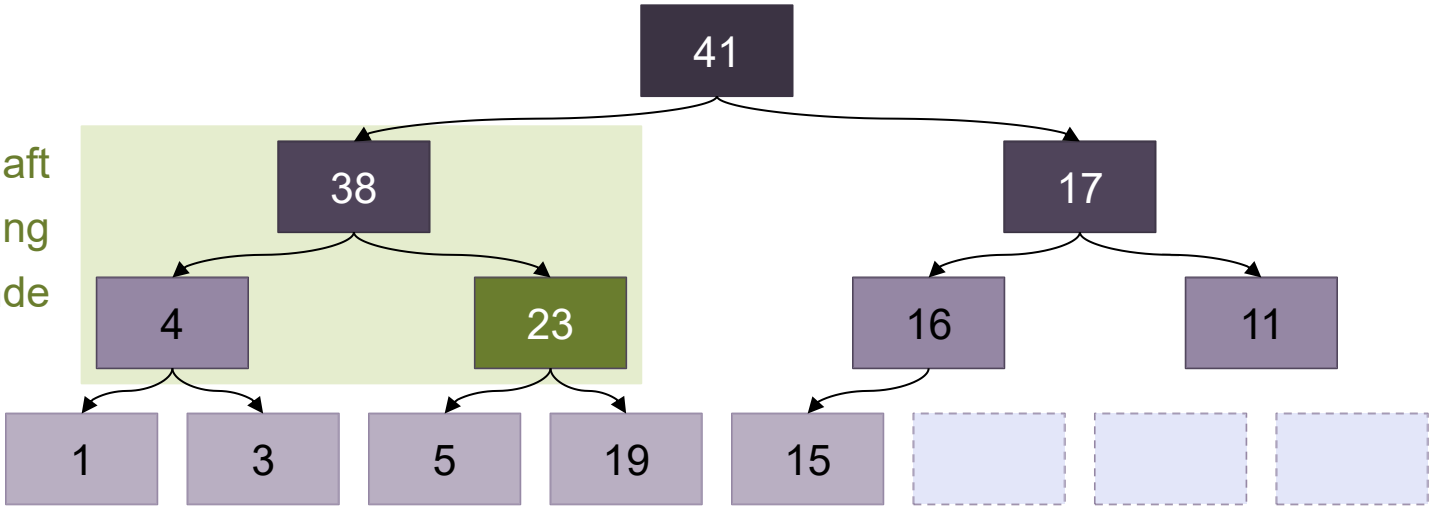


Heaps (9)

2. Schritt

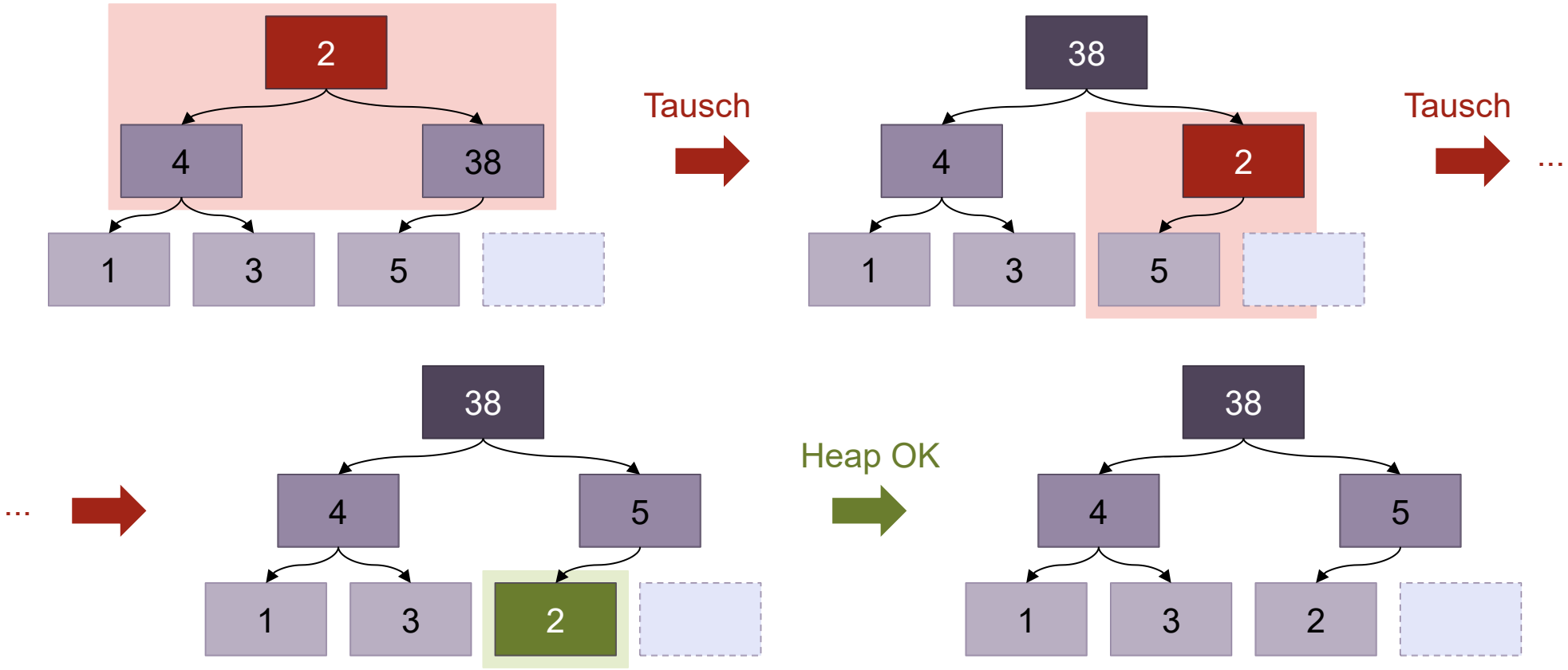


Heap-Eigenschaft
in Kind in Ordnung
→ Ende



Heaps (10)

weiteres Beispiel:



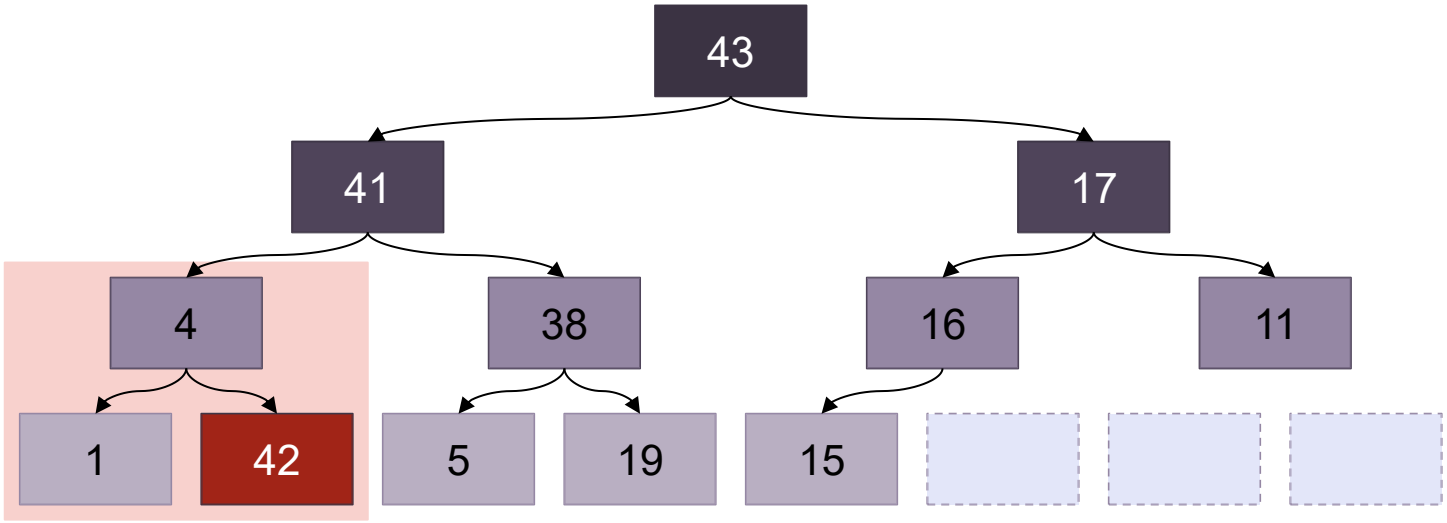
→ Rekursionsende spätestens bei den Blättern

→ leere Blätter werden ggf. ignoriert

Heaps (11)

Heapeigenschaft von unten nach oben herstellen

Heap-Eigenschaft
in Elternknoten von Blatt
durch Blatt verletzt



Gedankenspiel:

angenommen, **nur** in einem Blatt steht ein Element, welches die Heap-Eigenschaft des Elternknotens (und ggf. dem anderen Kind) zerstört, der restliche Heap ist OK

→ Wie stellt man die Heapeigenschaft so einfach wie möglich wieder her?

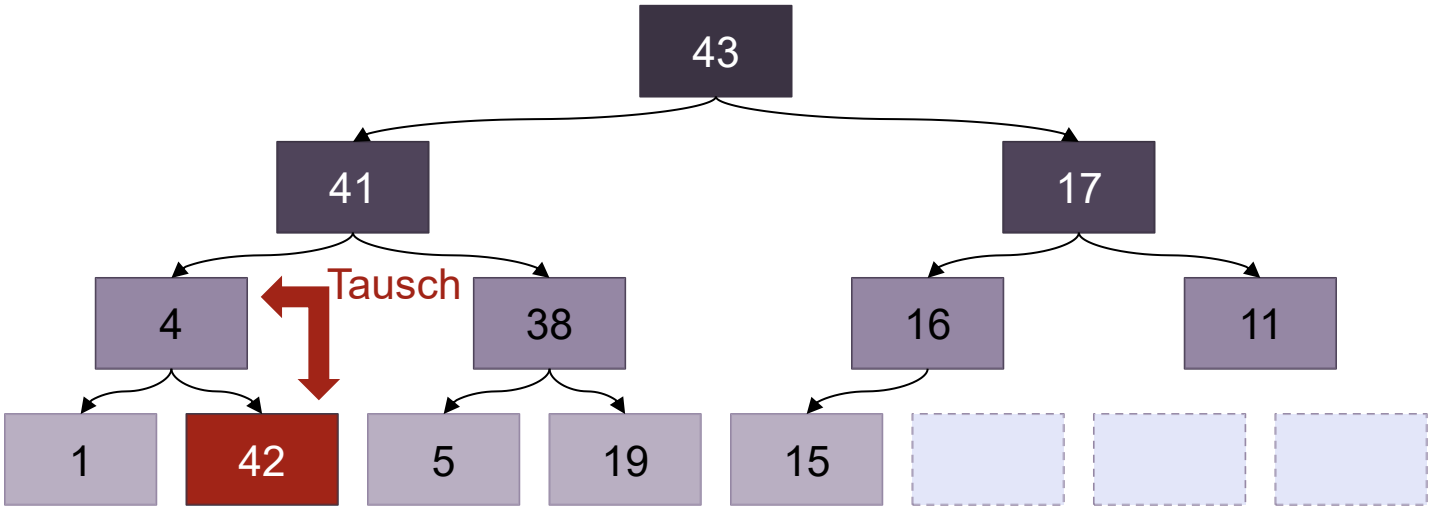
→ "*sift up*": repariere die Heapeigenschaft Stück für Stück in Richtung Wurzel durch Tausch mit und Rekursion zum Elternteil

→ höre auf, wenn Heapeigenschaft an einem Teilbaum schon erfüllt ist

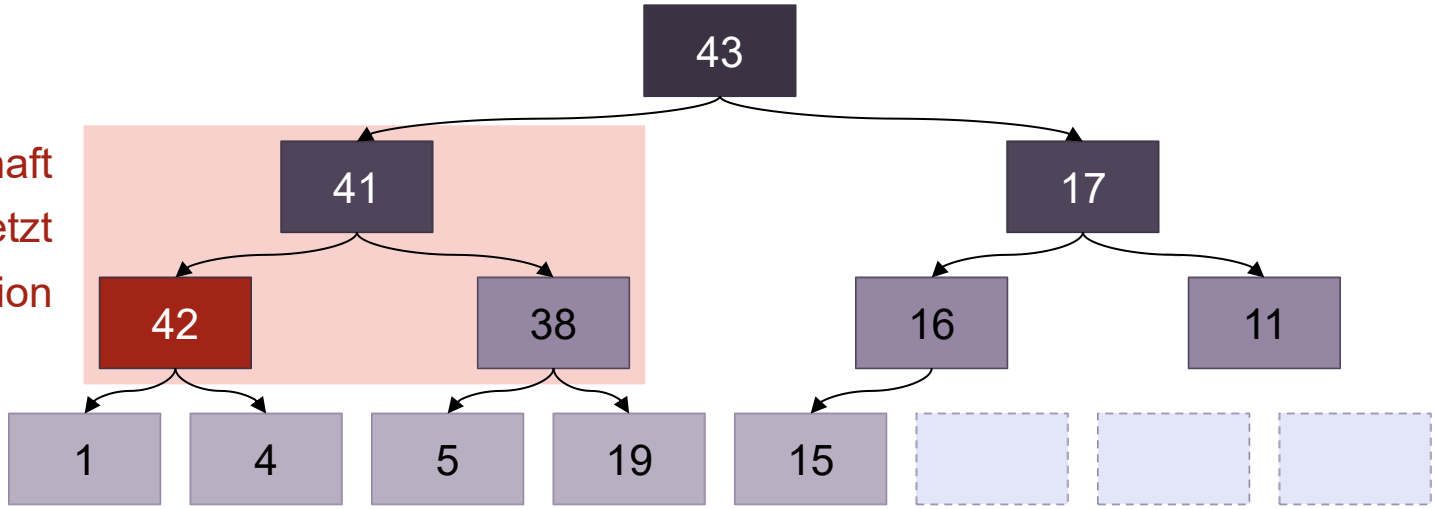
Heaps (12)

Beispiel:

1. Schritt

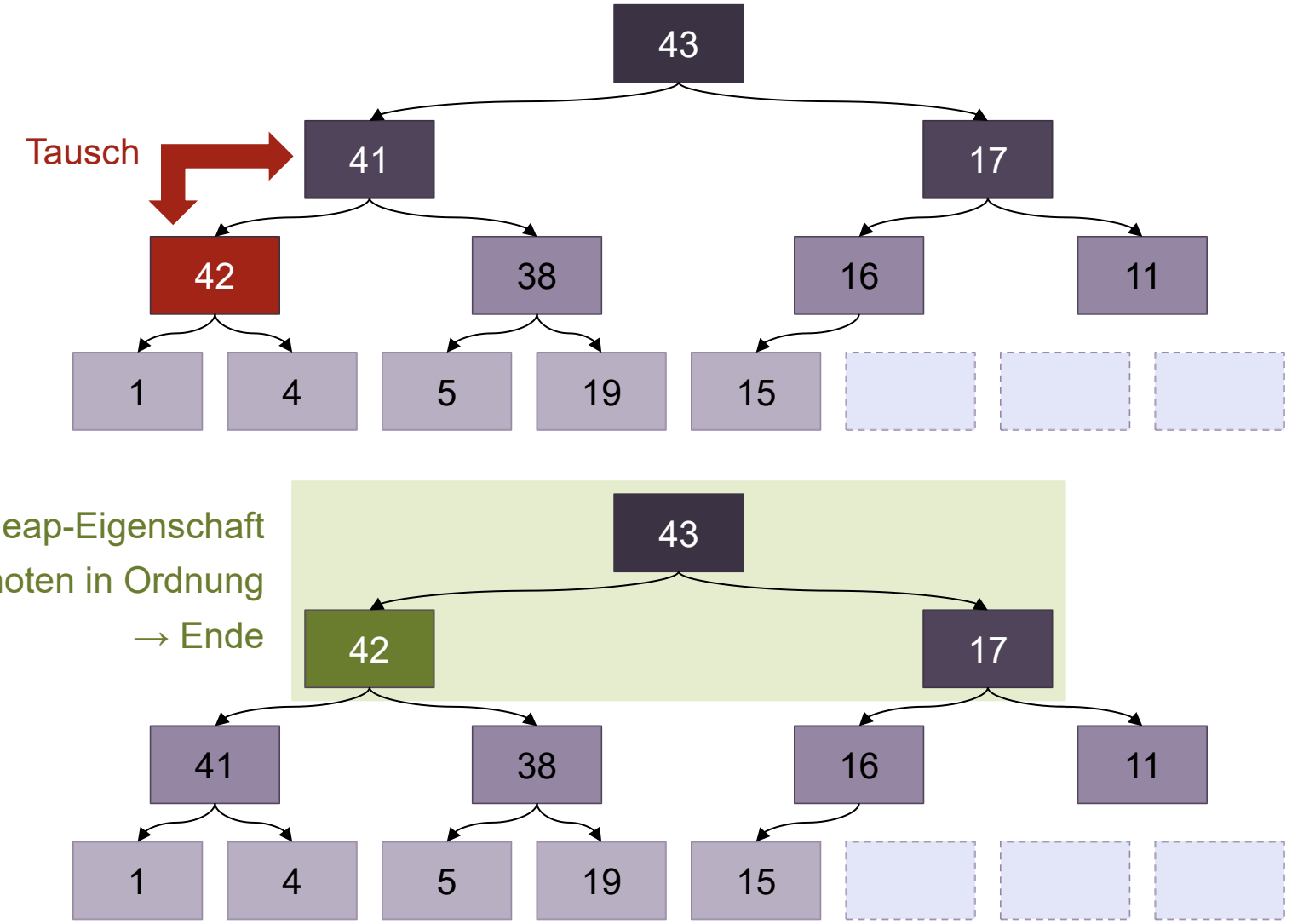


Heap-Eigenschaft in Elternknoten verletzt → Rekursion



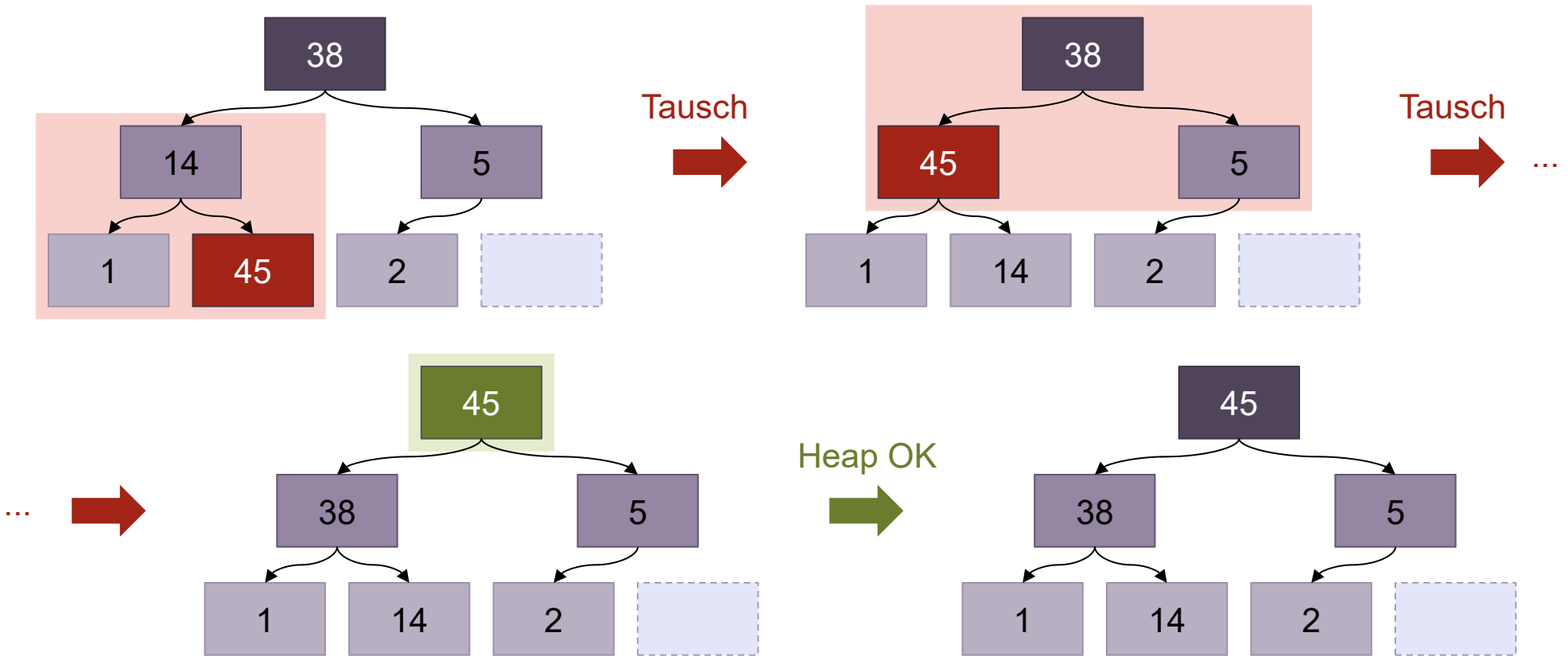
Heaps (13)

2. Schritt



Heaps (14)

weiteres Beispiel:



→ Rekursionsende spätestens bei der Wurzel

→ andere Spezialfälle gibt es hier nicht (vgl. sift-down: leere Blätter)

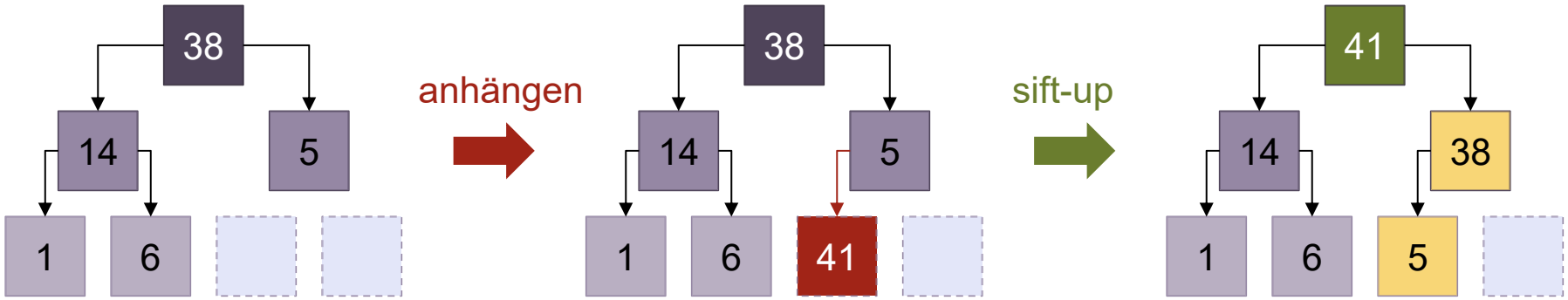
Heaps (15)

Einfügen im Heap

Einfügen unter Aktualisierung der Sortierung:

gegeben: bereits aufgebauter Heap, ein neues Datum

gesucht: korrekt um Datum erweiterter Heap



Vorgehen:

füge Datum am Ende des Heaps als neues Blatt an (ggf. in neuer Reihe)

→ die Heap-Eigenschaft ist womöglich auf einem Ast verletzt

→ stelle vom Blatt zur Wurzel die Heapeigenschaft mittels sift-up wieder her

→ es wird nur ein Ast des Heaps angefasst, Laufzeit O(log *n*) für *n* Elemente im Heap

Heaps (16)

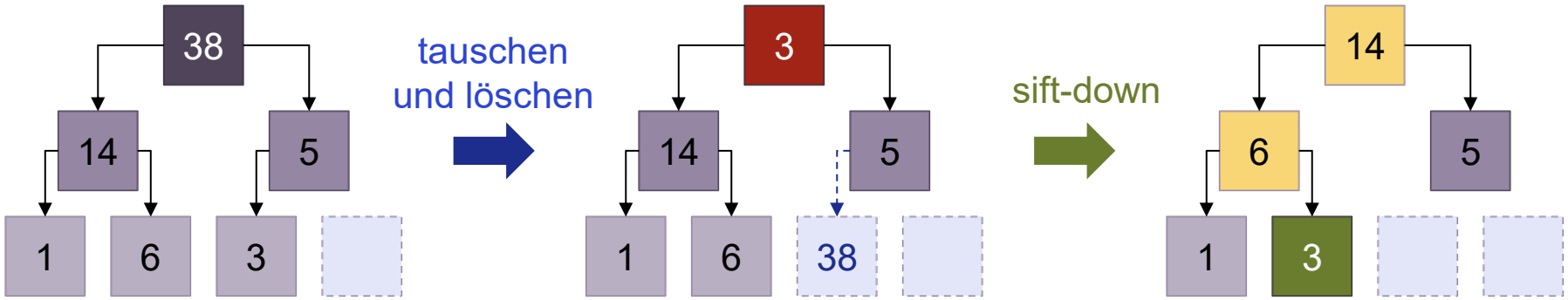
Löschen im Heap

Knoten zu löschen,
die nicht die Wurzel sind,
ist beim Heap schwierig!

Löschen unter Aktualisierung der Sortierung:

gegeben: bereits aufgebauter Heap

gesucht: korrekter Heap ohne Datum aus Wurzelknoten



Vorgehen:

tausche zu löschendes Wurzeldatum mit letztem Element und entferne dieses

→ die Heap-Eigenschaft ist auf einem Ast verletzt

→ stelle von der Wurzel zu den Blättern die Heapeigenschaft mittels sift-down wieder her

→ es wird nur ein Ast des Heaps angefasst, Laufzeit O(log *n*) für *n* Elemente im Heap

Heaps (17)

Array in Heap umwandeln

gegeben: Array mit Daten

gesucht: umsortiertes Array, das implizit einen Heap darstellt (vgl. Adapter für Arrays)

Vorgehen:

→ nehme an, das erste Arrayelement ist bereits ein (ein-elementiger) Heap

→ füge iterativ jeweils ein weiteres Arrayelement in den Heap ein (d. h. sift-up)

→ nach dem Einfügen des letzten Datum ist der Heap vollständig und korrekt

Beispiel:

hier als Max-Heap (wie in den vorherigen Beispielen),

Min-Heap analog (dann kleinere Elemente hochziehen)

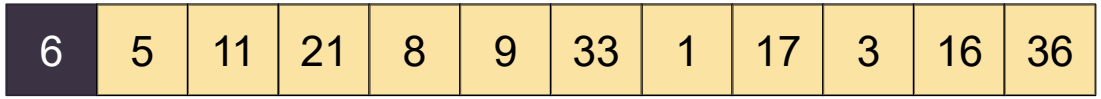
6	5	11	21	8	9	33	1	17	3	16	36

Array ist nicht sortiert

→ stückweise in Heap umwandeln

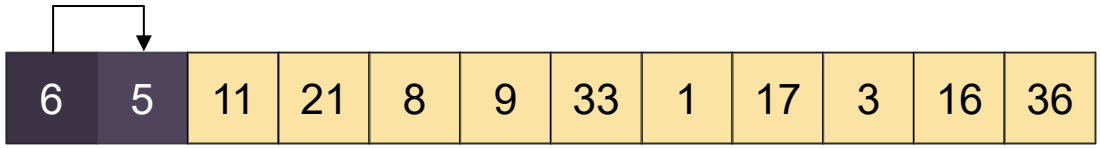
Heaps (18)

1. Element zu Heap hinzufügen:



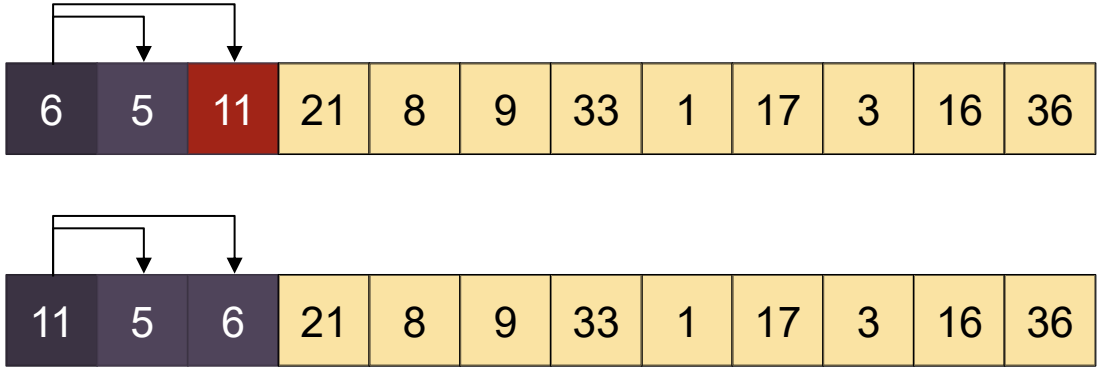
Heap-Eigenschaft schon erfüllt
→ keine Aktion nötig

2. Element zu Heap hinzufügen:



Heap-Eigenschaft schon erfüllt
→ keine Aktion nötig

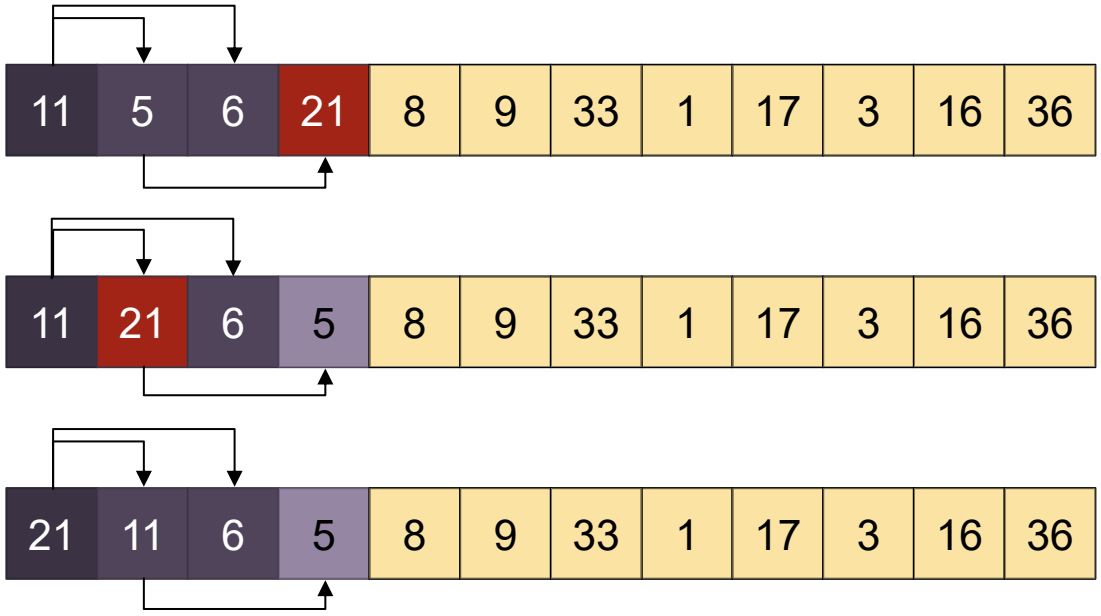
3. Element zu Heap hinzufügen:



Heap-Eigenschaft nicht erfüllt
→ mit Sift-Up wiederherstellen

Heaps (19)

4. Element zu Heap hinzufügen:

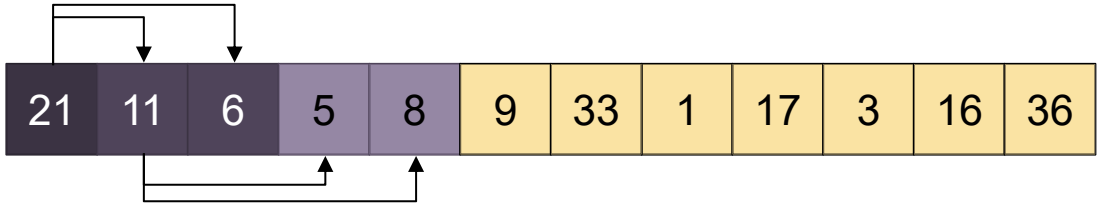


Heap-Eigenschaft nicht erfüllt

→ mit Sift-Up wiederherstellen

→ hier zwei Mal tauschen nötig

5. Element zu Heap hinzufügen:

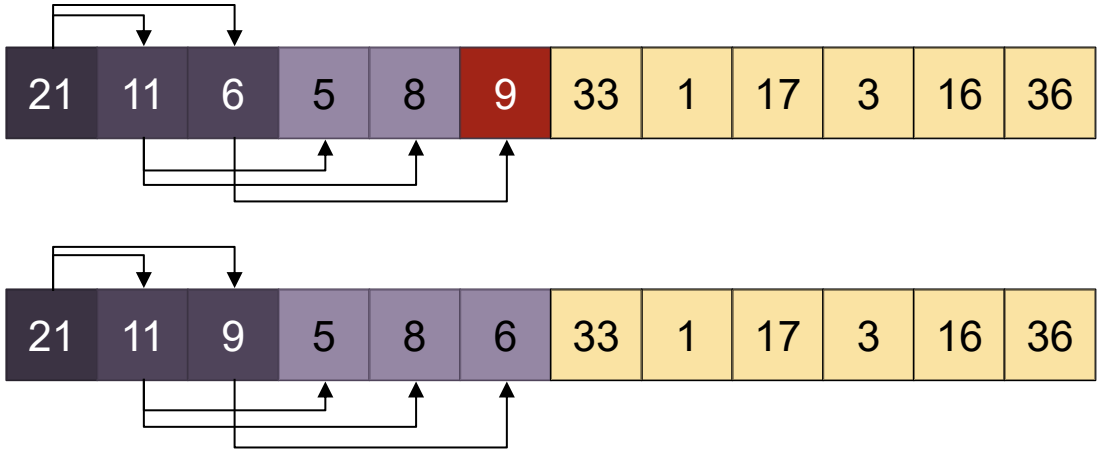


Heap-Eigenschaft schon erfüllt

→ keine Aktion nötig

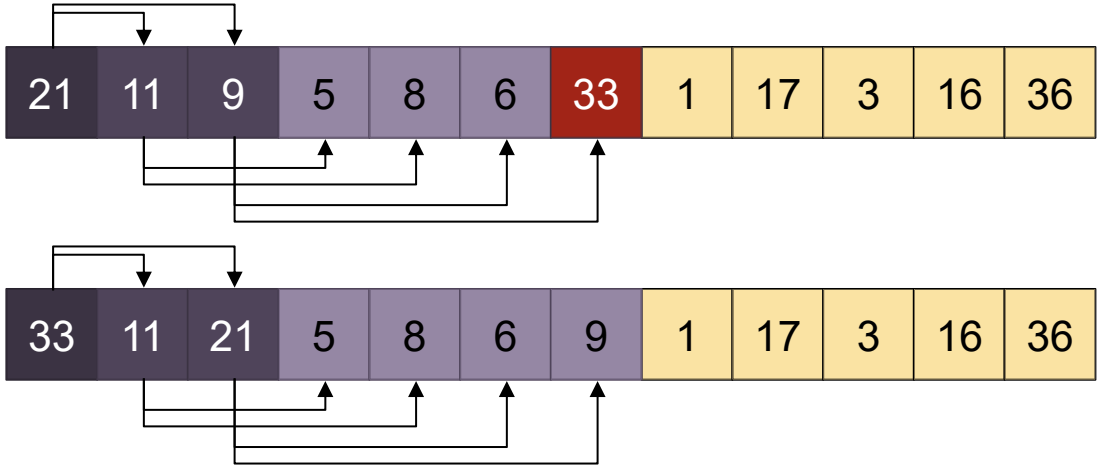
Heaps (20)

6. Element zu Heap hinzufügen:



Heap-Eigenschaft nicht erfüllt
→ mit Sift-Up wiederherstellen
→ hier ein Mal tauschen nötig

7. Element zu Heap hinzufügen:

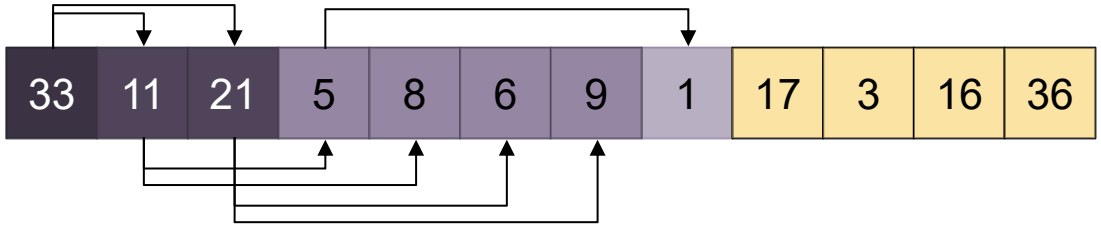


Heap-Eigenschaft nicht erfüllt
→ mit Sift-Up wiederherstellen
→ hier zwei Mal tauschen nötig

ab jetzt: nötige Zwischenschritte für Tauschen nicht mehr gezeigt

Heaps (21)

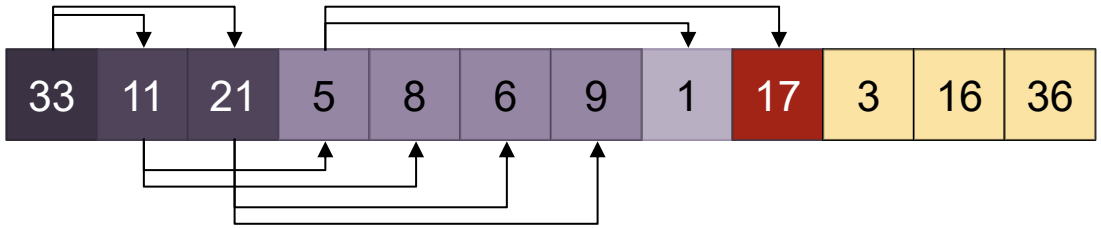
8. Element zu Heap hinzufügen:

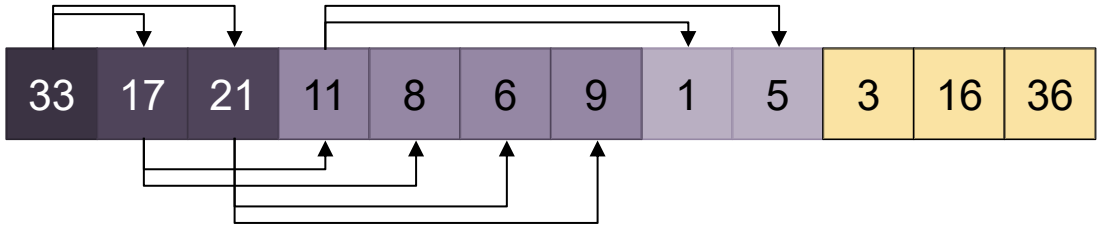


Heap-Eigenschaft schon erfüllt

→ keine Aktion nötig

9. Element zu Heap hinzufügen:





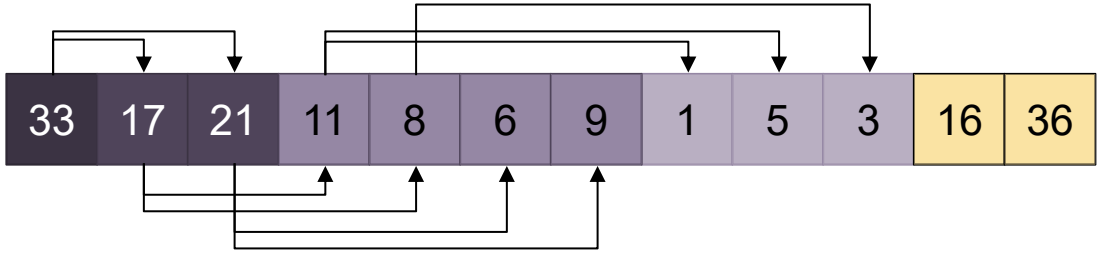
Heap-Eigenschaft nicht erfüllt

→ mit Sift-Up wiederherstellen

→ hier zwei Mal tauschen nötig

Heaps (22)

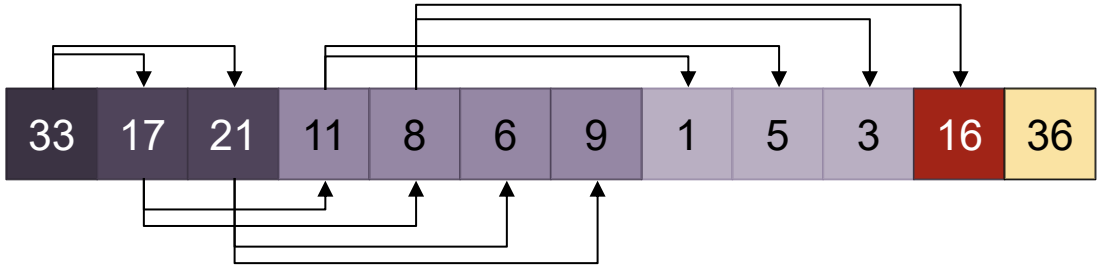
10. Element zu Heap hinzufügen:

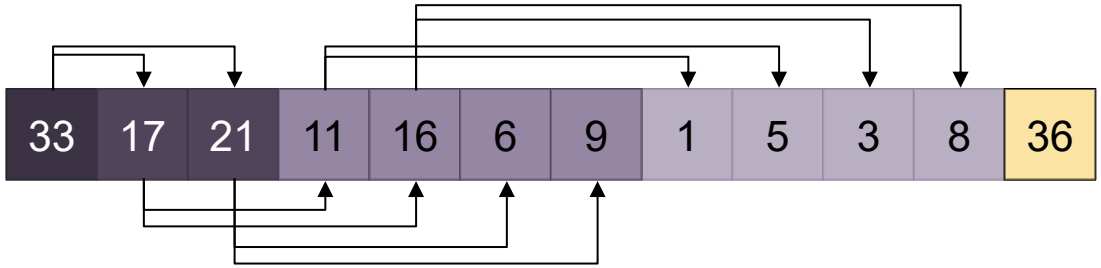


Heap-Eigenschaft schon erfüllt

→ keine Aktion nötig

11. Element zu Heap hinzufügen:





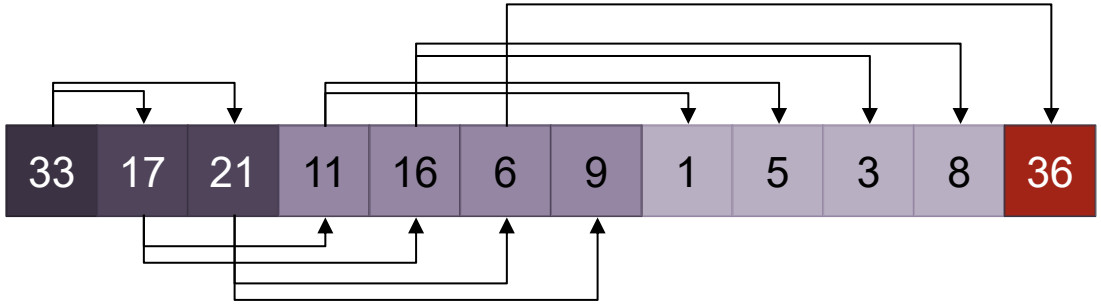
Heap-Eigenschaft nicht erfüllt

→ mit Sift-Up wiederherstellen

→ hier ein Mal tauschen nötig

Heaps (23)

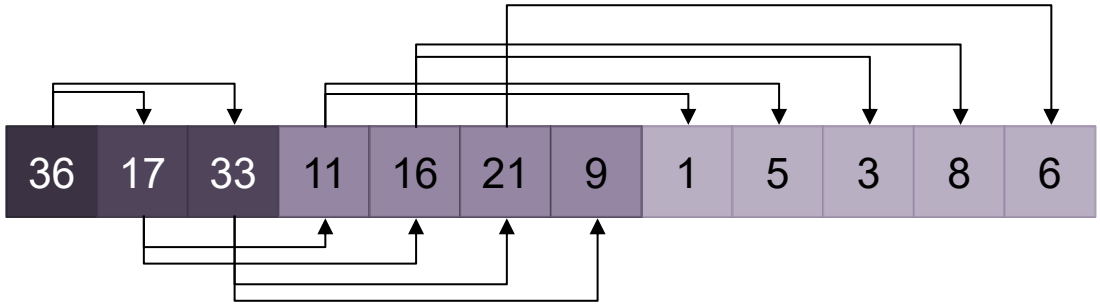
12. Element zu Heap hinzufügen:



Heap-Eigenschaft nicht erfüllt

→ mit Sift-Up wiederherstellen

→ hier drei Mal tauschen nötig



Array vollständig

zu Heap umsortiert

Bewertung:

- Laufzeit O(*n* log *n*)
- das Verfahren läuft *in-situ* (lat. für "am Platz", engl. *in-place*),
  d. h. ohne zusätzlichen Speicherbedarf für Daten

Heaps (24)

Heap-Sort

gegeben: Array mit zu sortierenden Daten

gesucht: sortiertes Array mit den Daten

Vorgehen:

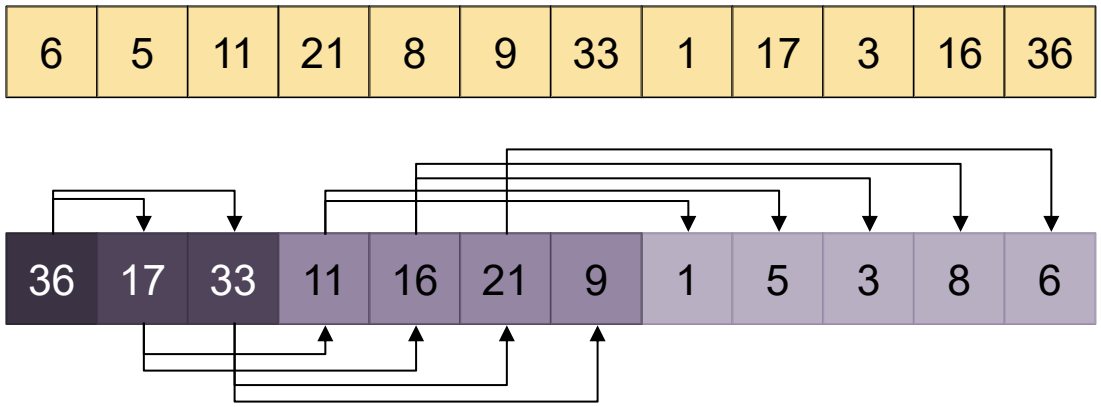
→ wandle Array in Heap um (vgl. vorherige Folien)

→ lösche wiederholt die Wurzel, bis der Heap leer ist

→ gelöschte Elemente werden durch das Löschen der Wurzel am Ende des Array abgelegt

→ Array enthält am Ende die aufsteigend sortierten Daten

Beispiel:

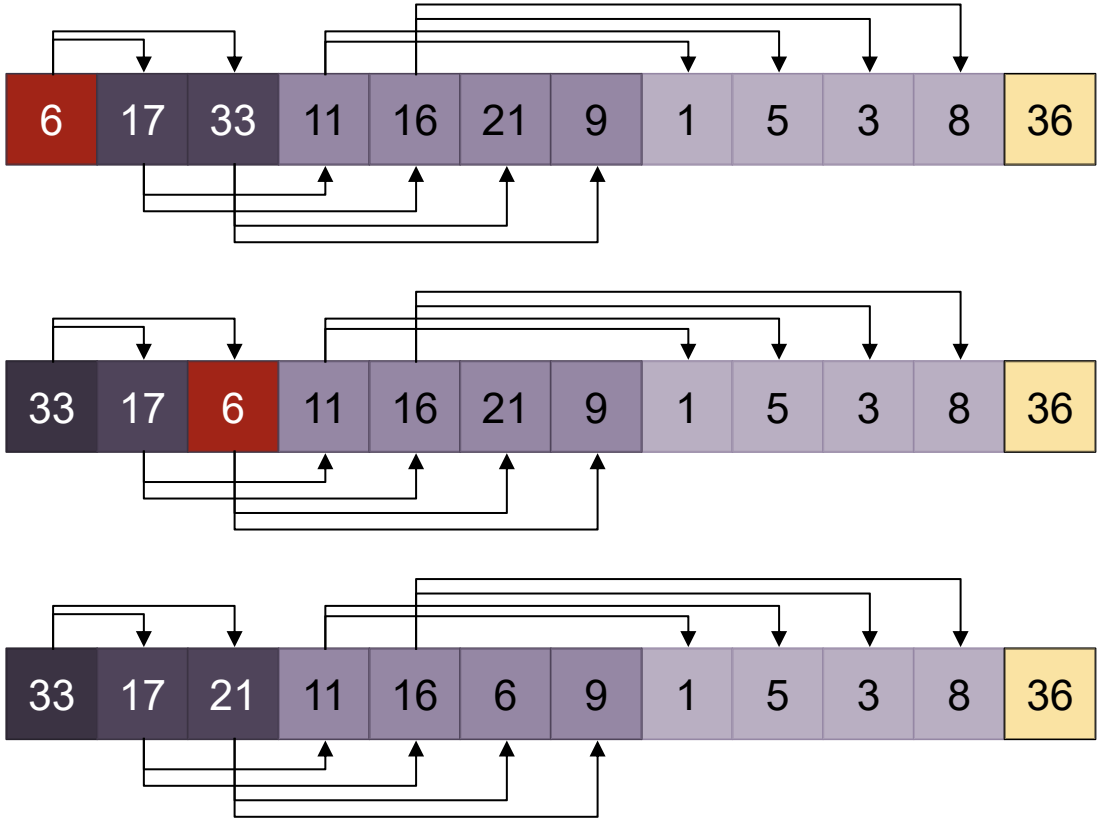


unsortiertes Array
als Eingabe

Array zu Heap umgewandelt
(vgl. vorheriges Beispiel)

Heaps (25)

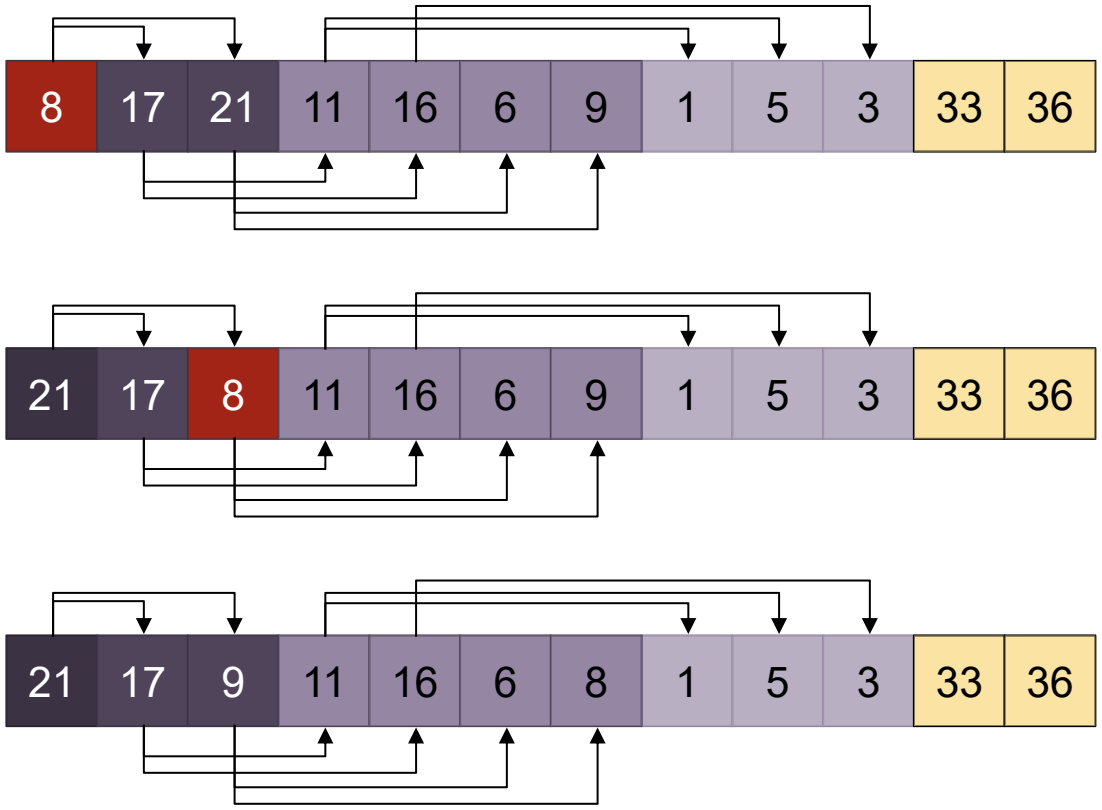
1. Element aus Heap entfernen:



Löschen der Wurzel
→ Wurzel mit letztem Array-Eintrag tauschen und herausnehmen
→ Heap-Eigenschaft verletzt
→ durch sift-down wiederherstellen (hier zwei Mal nötig)

Heaps (26)

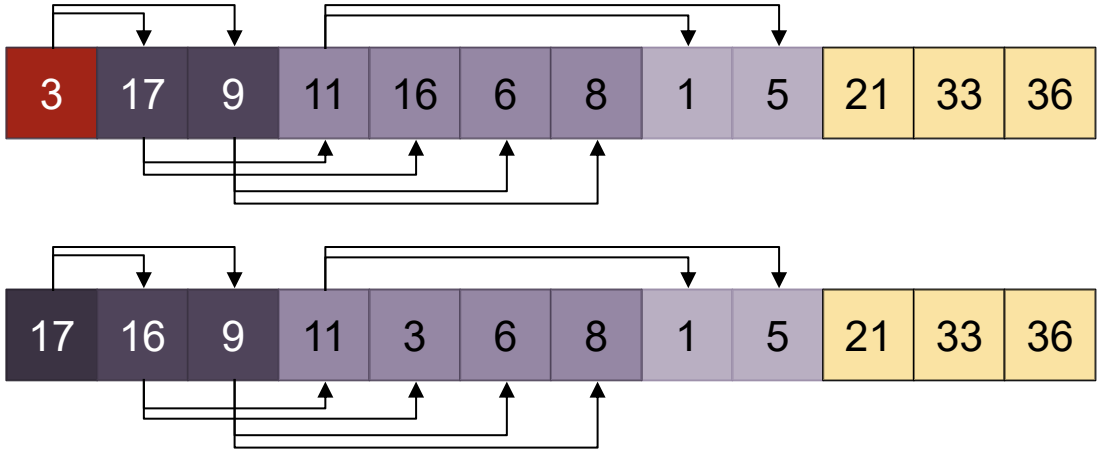
2. Element aus Heap entfernen:



Löschen der Wurzel, sift-down,
hier zwei Mal tauschen

Heaps (27)

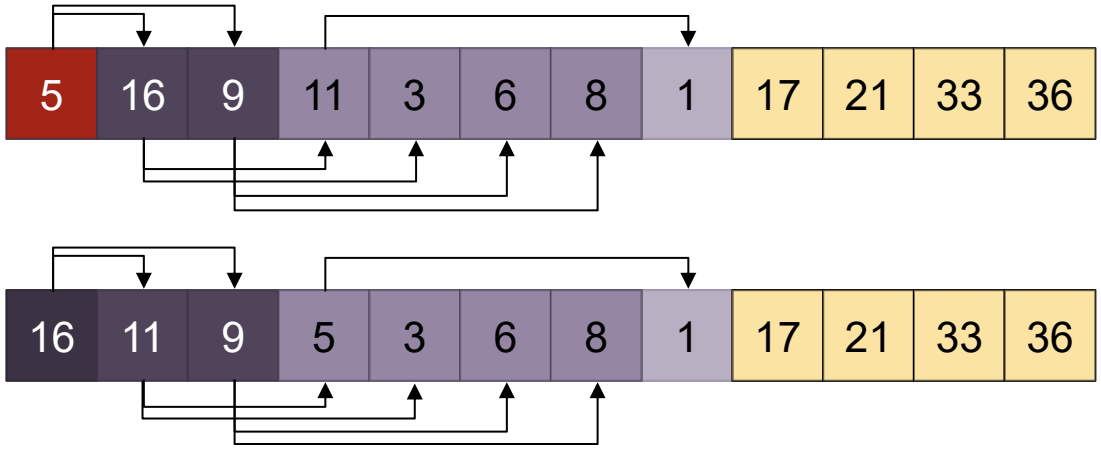
3. Element aus Heap entfernen:



Löschen der Wurzel, sift-down, hier zwei Mal tauschen

ab jetzt: nötige Zwischenschritte für Tauschen nicht mehr gezeigt

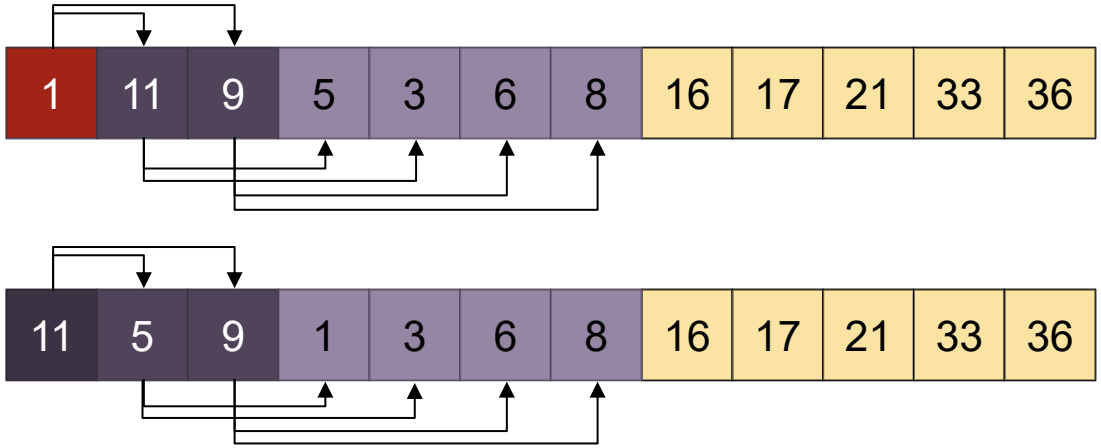
4. Element aus Heap entfernen:



Löschen der Wurzel, sift-down, hier zwei Mal tauschen

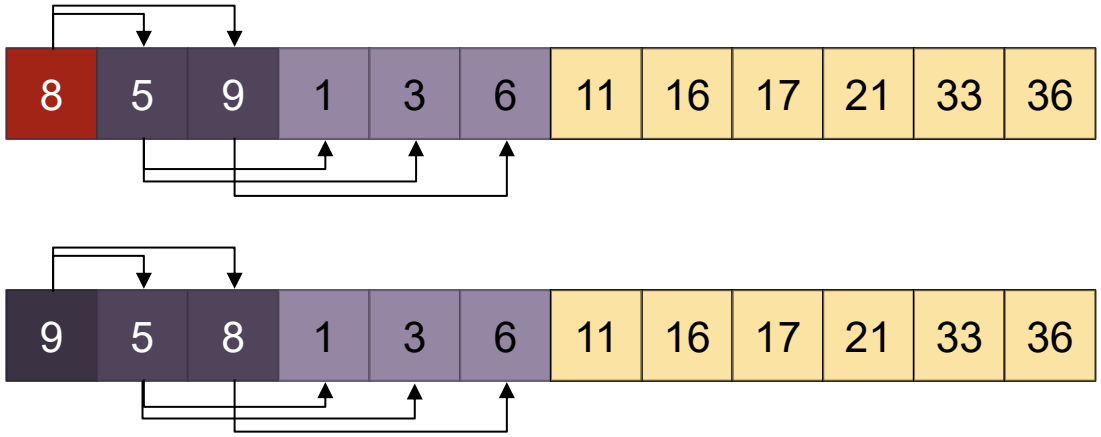
Heaps (28)

5. Element aus Heap entfernen:



Löschen der Wurzel, sift-down, hier zwei Mal tauschen

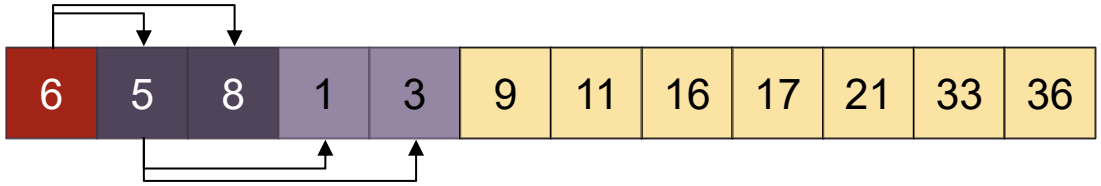
6. Element aus Heap entfernen:

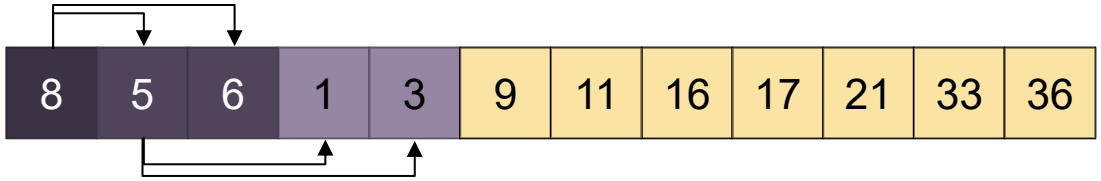


Löschen der Wurzel, sift-down, hier ein Mal tauschen

Heaps (29)

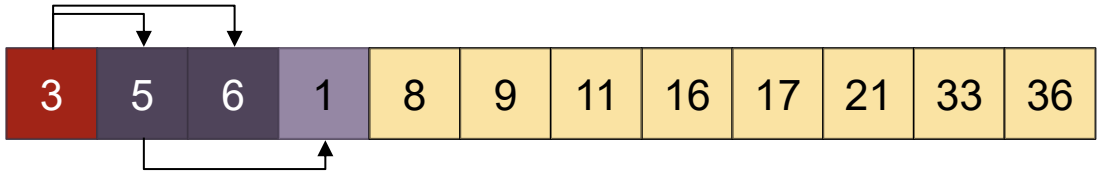
7. Element aus Heap entfernen:

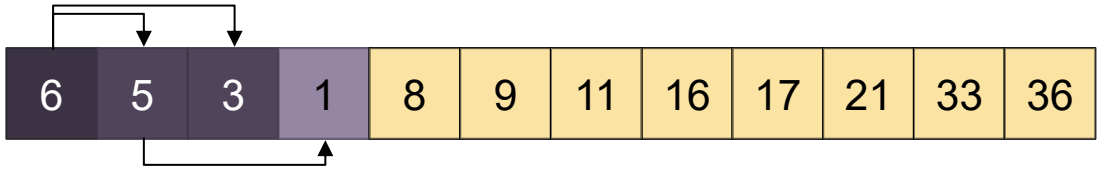




Löschen der Wurzel, sift-down, hier ein Mal tauschen

8. Element aus Heap entfernen:

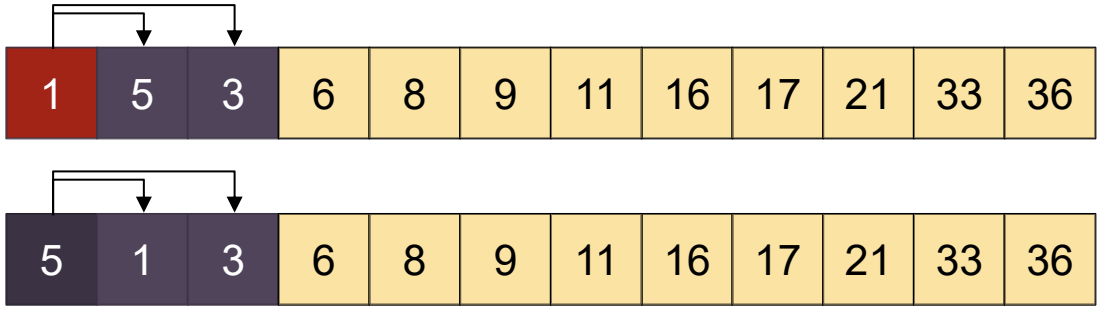




Löschen der Wurzel, sift-down, hier ein Mal tauschen

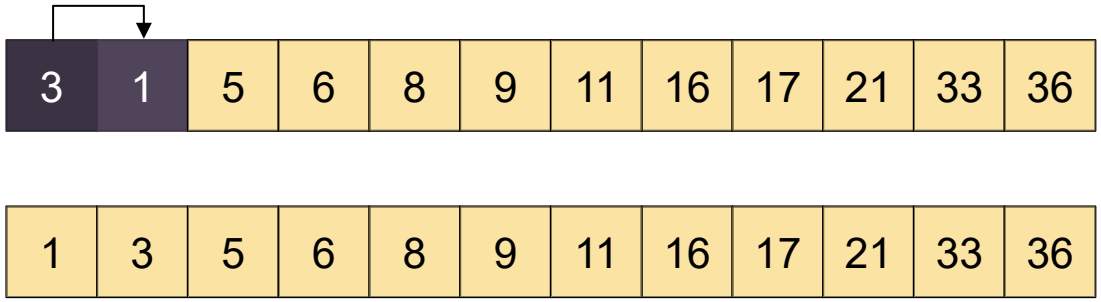
Heaps (30)

9. Element aus Heap entfernen:



Löschen der Wurzel, sift-down,
hier ein Mal tauschen

10. und 11. Element aus Heap entfernen:



Löschen der Wurzel, sift-down,
hier tauschen nicht nötig

Array vollständig
zu Heap umsortiert

Bewertung:

- Laufzeit O(n log n)
- Verfahren läuft in-situ

Vortragsübersicht

6 – Sequentielle Container und Adapter

- Arrays
- Listen
- Heaps
- Suche in Arrays
- Stacks und Queues

Suche in Arrays (1)

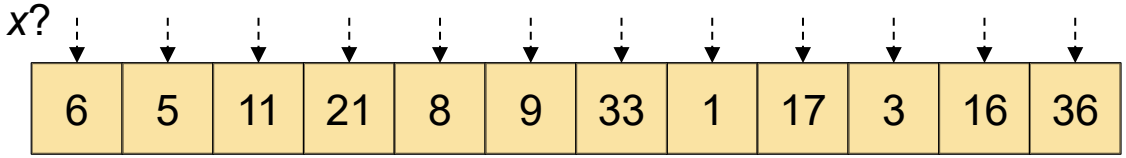
Werte in Arrays finden

häufiger Anwendungsfall: ist Wert *x* schon in Datenstruktur (z. B. Array) enthalten

gegeben: Container *C*, zu findender Wert *x*
gesucht: ist *x* in *C* enthalten, falls ja: wo?

konkreterer Anwendungsfall:

Container *C* ist ein Array



naive Lösung:

über die Werte in *C* ist weiter nichts bekannt
→ gehe alle Werte *c* in *C* durch und prüfe, ob *c* gleich *x* ist
→ Laufzeit O(*n*)
→ ohne zusätzliche Annahmen geht das nicht schneller,
aber vielleicht mit zusätzlichen Annahmen?

Suche in Arrays (2)

bessere Lösung mit Zusatzannahmen:

- das Array ist in sinnvoller Reihenfolge (z. B. aufsteigende Werte mit Heap-Sort wie zuvor) sortiert
- *divide-and-conquer*-Ansatz (dt. Teilen-und-Herrschen-Ansatz):
  teile ein großes Problem in mehrere, leichtere Teilprobleme
  und bestimme aus deren (Teil-)Lösungen die Gesamtlösung
- insbesondere Vorgehen mit *Intervallteilung*:
  → zerlege die zu prüfenden Indizes entlang eines *Pivots* (Angelpunkt)
  in zwei separate Mengen, prüfe beide Teilmengen rekursiv
  → je nach Pivot und zu findendem Eintrag geht das für eine Menge schnell,
  weil das gesuchte Element dort nicht liegen kann
  → Pivotwahl typischerweise Mitte des Arrays (alternativ: zufällig)

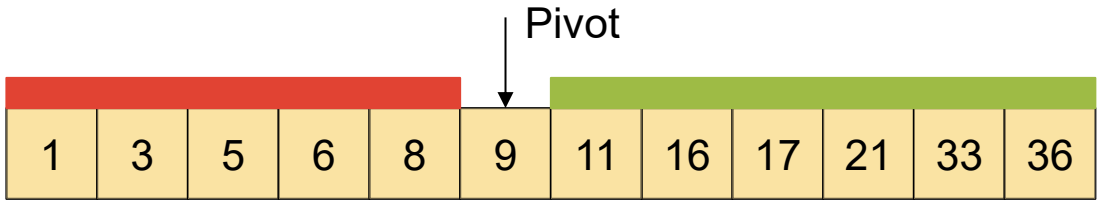
Beispiel:

Ist der Wert 11 in dem nachfolgenden Array enthalten?

1	3	5	6	8	9	11	16	17	21	33	36

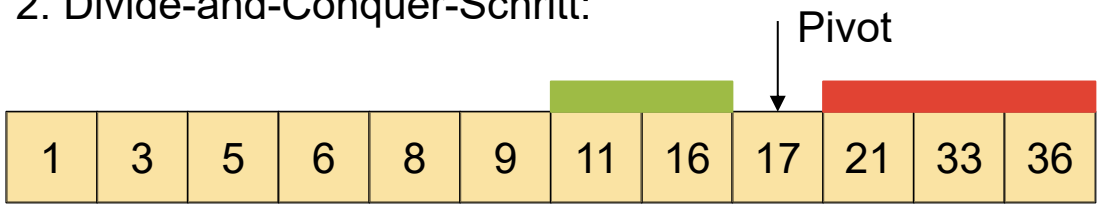
Suche in Arrays (3)

1. Divide-and-Conquer-Schritt:



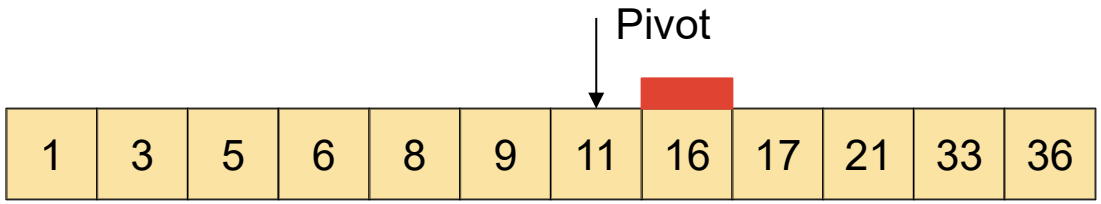
wähle Pivot als mittleres Element,
Index ist *abrunden*((11 + 0) / 2) = 5
→ 11 > 9, d. h. wenn 11 drin ist,
liegt es in der rechten Hälfte,
nicht in der linken Hälfte

2. Divide-and-Conquer-Schritt:



wähle Pivot als mittleres Element,
Index ist *abrunden*((11 + 6) / 2) = 8
→ 11 > 9, d. h. wenn 11 drin ist,
liegt es in der linken Hälfte,
nicht in der rechten Hälfte

3. Divide-and-Conquer-Schritt:



wähle Pivot als mittleres Element,
Index ist *abrunden*((6 + 7) / 2) = 6
→ 11 = 11, d. h. Wert = Pivot
→ Wert gefunden, Ende

→ Laufzeit: O(log *n*)

Vortragsübersicht

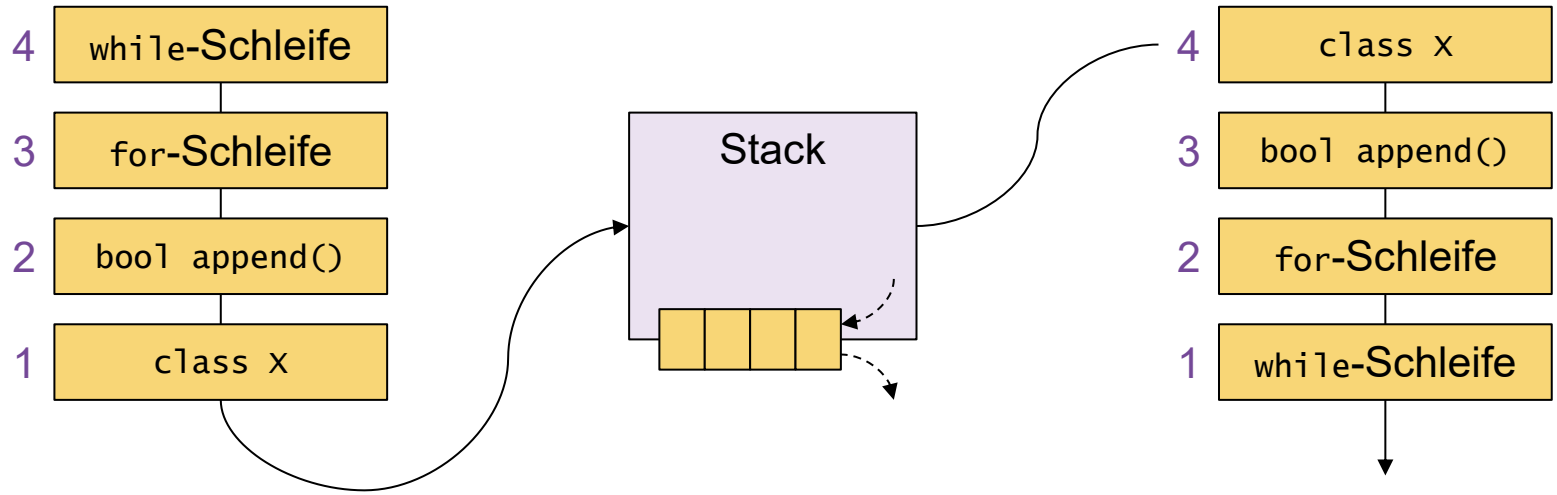
6 – Sequentielle Container und Adapter

- Arrays
- Listen
- Heaps
- Suche in Arrays
- Stacks und Queues

Stacks und Queues (1)

Stack

Nummerierung ist implizit, nicht Teil der Daten!



der *Stack* (selten *Last-In-First-Out-Queue*, kurz *LIFO-Queue*, dt. *Keller*, *Stapel*)
ist ein Container, der Elemente in umgekehrter Einfügereihenfolge herausgibt
(d. h. zuletzt eingefügte Elemente werden als erstes herausgegeben)

→ Praxisbeispiel: Tellerstapel im Geschirrschrank

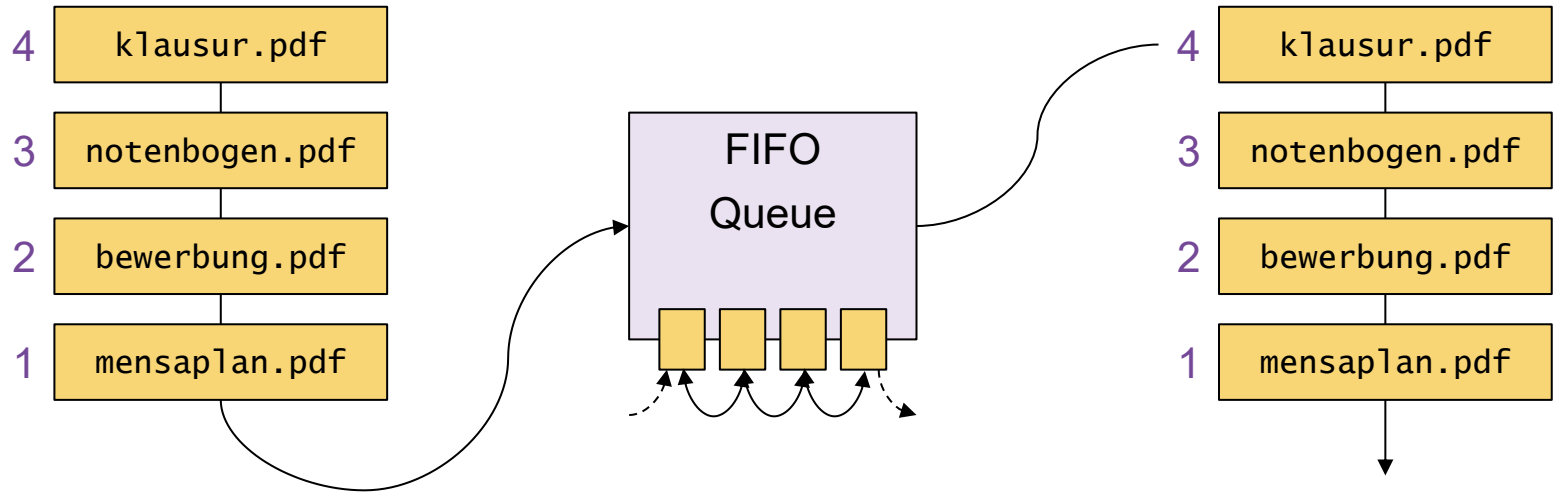
→ IT-Anwendung: Abarbeiten von Daten mit geschachteltem Kontext

→ Umsetzung: typischerweise mittels Array und `push_back()`, `pop_back()`
(Liste ist aber auch möglich)

Stacks und Queues (2)

FIFO-Queue

Nummerierung ist implizit, nicht Teil der Daten!



die *First-In-First-Out-Queue* (kurz *FIFO-Queue*, *Queue*, dt. *Warteschlange*)
ist ein Container, der Elemente in Reihenfolge des Hineingebens wieder herausgibt

→ Praxisbeispiel: Warteschlange an der Supermarktkasse

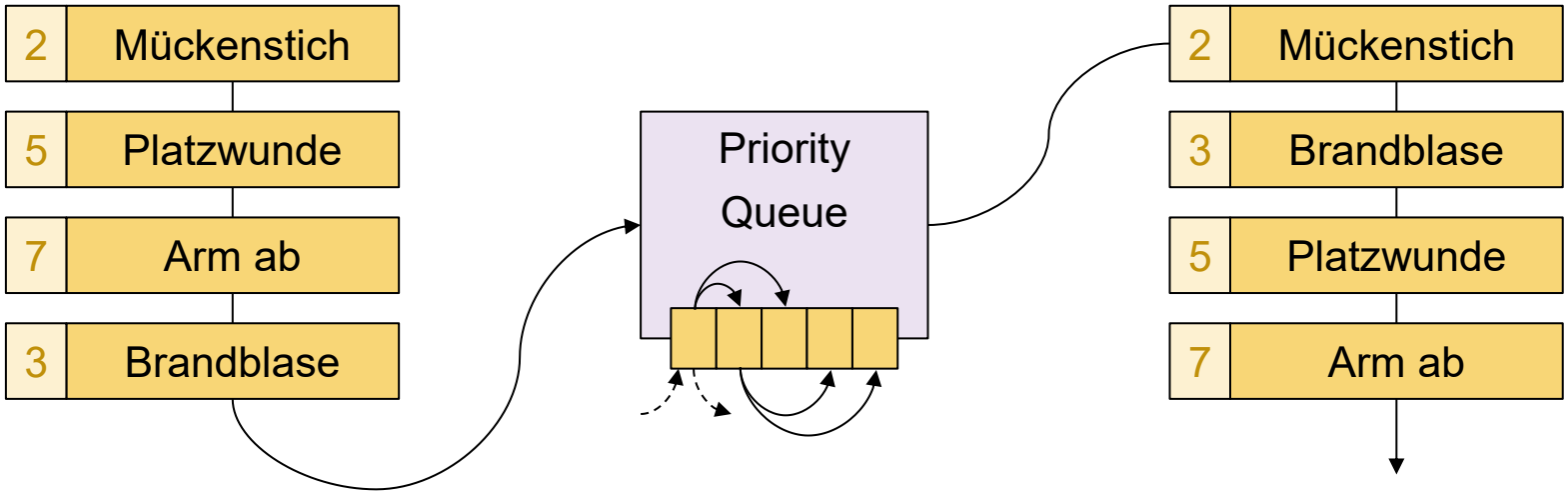
→ IT-Anwendung: Zwischenspeicher einer *Datenverarbeitungs-Pipeline*
(d. h. Eingabedaten *seriell*, also hintereinander, abarbeiten)

→ Umsetzung: typischerweise mittels Liste und `push_front()`, `pop_back()`
(Array ist aber auch möglich)

Stacks und Queues (3)

Priority-Queue

Nummerierung ist Teil der Daten!



die *Priority-Queue* (dt. *Prioritätswarteschlange*) ist ein Container,
der Elemente in Reihenfolge einer Priorität wieder hinausgibt

→ Praxisbeispiel: Warteschlange in der Notaufnahme vom Krankenhaus

→ IT-Anwendung: Aufgaben mit sowie gemäß Priorisierung abarbeiten
(z. B. UI-Ereignisse, Fahrzeugassistenzereignisse)

→ Umsetzung: typischerweise mit Heap sortiert nach gegebener Priorität,
mit Trennung zwischen Priorität und weiteren Elementdaten

Zusammenfassung

Grundlagen

- Array is sequentieller, dicht gepackter Container mit impliziten Verweisen
  → Zugriff schnell, wenig Speicheroverhead
  → Einfügen und Löschen relativ langsam und invalidiert Iteratoren
- Liste ist sequentieller, lose gepackter Container mit expliziten Verweisen
  → Zugriff langsam, viel Speicheroverhead
  → Einfügen und Löschen relativ schnell und erhält Iteratoren

Fortgeschrittene Container

- Heap speichert nach Abstufung geordnete Elemente
  → schnelles Auslesen und Sortieren
  → Realisierung über Array und Herstellen einer Heap-Eigenschaft
- zusätzliche Container als Adapter:
  FIFO-Queue auf Liste, Priority Queue auf Heap, Stack auf Array